

PART 4

Pythonic Techniques & Core Power Tools

Comprehensions | Built-in Functions | Pythonic Techniques
Collection Tools | Coding Patterns | Cheat Sheet
40 Conceptual Questions | 40 Output Questions | 40 Coding Problems
Complete Solutions | Interview Tricks

Beginner → Intermediate → Advanced

Target: Python interviews, DSA, competitive programming, and clean everyday code.
Every output in the “Output Questions” section was produced by actually running the code (Python 3.12), and every coding solution was tested before printing.

Contents

How to Use This Part	3
Learning path inside this part	3
Corrections and upgrades to your source notes	3
1. Comprehensions	5
2. Built-in Functions	9
2.1 Aggregate and numeric functions	9
2.2 Ordering and iteration functions	12
2.3 Logic and functional functions	15
2.4 Introspection and character functions	17
3. Pythonic Techniques	20
4. Collection Tools	27
5. Common Coding Patterns	32
6. Python Shortcuts Cheat Sheet	41
6.1 Comprehensions and built-ins	41
6.2 Assignment, unpacking, sorting	41
6.3 Collections module and friends	42
6.4 Complexity quick table	42
6.5 Ten lines to memorise	42
7. Practice Set	43
7.1 Forty Conceptual Questions	43
7.2 Forty Output Questions	44
7.3 Forty Coding Problems	47
8. Complete Solutions	50
8.1 Conceptual Answers	50
8.2 Output Answers (every output was generated by running the code)	52
8.3 Coding Solutions (every solution was tested)	59
9. Interview Tricks	69
9.1 Before you type	69
9.2 Tool-choice reflexes	69
9.3 The classic traps (know all of them)	69
9.4 Phrases that impress	70
9.5 Practice routine	70

How to Use This Part

Parts 1-3 built the base: variables, types, control flow, functions, and references. **Part 4 is where you start writing Python the way experienced Python programmers do** - shorter, clearer, and faster - and where you pick up the standard-library tools that turn many $O(n^2)$ interview solutions into $O(n)$.

Learning path inside this part

Level	What you can do after it
Beginner	Read and write comprehensions, use the everyday built-ins, swap/unpack values, sort with <code>key=</code> .
Intermediate	Choose between <code>Counter</code> , <code>defaultdict</code> , <code>deque</code> , <code>heapq</code> , <code>bisect</code> ; recognise frequency / grouping / two-sum / stack / queue / heap patterns.
Advanced	Know the traps (lazy iterators, stable sort, banker's rounding, shared references, heap tie-breaks) and the complexity of every tool.

Each technique uses the same 7-point card: **(1)** what it does, **(2)** why it is useful, **(3)** syntax, **(4)** simple example, **(5)** before vs Pythonic, **(6)** common mistake, **(7)** interview use. In code, `# ->` marks a value or printed output; **O(...)** is average / amortised CPython cost; **C++ contrast** boxes are short reminders for anyone coming from C++.

Corrections and upgrades to your source notes

Your notes were a strong base. While reorganising them I fixed or tightened the following points - several of them are exactly the kind of detail interviewers probe.

Topic in source notes	Correction / upgrade
<code>list(set(arr))</code> to remove duplicates	Works, but destroys the original order (sets are unordered). To keep first-seen order use <code>list(dict.fromkeys(arr))</code> . Dicts preserve insertion order (guaranteed since Python 3.7).
<code>Counter(arr)</code> shown as <code>{3:3, 2:2, 1:1}</code>	The real display is <code>Counter({3: 3, 2: 2, 1: 1})</code> . A <code>Counter</code> is a dict subclass ; a missing key returns 0 instead of raising <code>KeyError</code> .
Two Sum with a set	A set is fine for <i>*does a pair exist?*</i> but cannot give indices . To return indices store value <code>-></code> index in a dict .
<code>round()</code> listed as plain "rounding"	Python 3 uses banker's rounding (round-half-to-even): <code>round(2.5) == 2</code> , <code>round(3.5) == 4</code> , <code>round(0.5) == 0</code> .
<code>map</code> , <code>filter</code> , <code>zip</code> , <code>reversed</code> , <code>enumerate</code>	All return lazy iterators (one-shot). <code>print(map(...))</code> shows an object, and a second loop over the same iterator is empty. Wrap in <code>list()</code> when you need a list.
<code>s.find(x)</code> for checking	Returns -1 when not found, and -1 is truthy, so if <code>s.find(x):</code> is a bug. Use <code>x in s</code> .
max heap by negating	Correct for numbers. For strings or objects negate is impossible - use a tuple key or wrapper. <code>heapq</code> has no public max-heap API .
<code>bisect_left</code> only	Also know <code>bisect_right</code> (insert <i>*after*</i> equal items) and <code>insort</code> . <code>Bisect</code> requires an already sorted list.
<code>sum</code> , <code>max</code> , <code>min</code> on empty input	<code>sum([]) == 0</code> , but <code>max([])</code> / <code>min([])</code> raise <code>ValueError</code> unless you pass <code>default=</code> .
<code>any</code> / <code>all</code> behaviour	<code>any([])</code> is <code>False</code> and <code>all([])</code> is <code>True</code> (vacuous truth). A classic trick question.
<code>(l + r) // 2</code> for binary search	Fine in Python because ints never overflow. (In C++ you would write <code>l + (r - l) / 2</code> .)
<code>x // y</code> and <code>x % y</code>	Python floors : <code>-7 // 2 == -4</code> and <code>-7 % 2 == 1</code> . C++ truncates toward zero (<code>-7 / 2 == -3</code> , <code>-7 % 2 == -1</code>). Python's <code>%</code> result always has the sign of the divisor.
Comprehension variable scope	In Python 3 the loop variable of a comprehension does not leak into the surrounding scope (unlike Python 2 list comprehensions).

1. Comprehensions

A **comprehension** builds a collection from an iterable in a single expression. It replaces the "create empty container, loop, append" ritual. Four flavours exist:

Kind	Brackets	Result
List comprehension	[expr for x in it]	list
Set comprehension	{ expr for x in it }	set (unique, unordered)
Dict comprehension	{ k: v for x in it }	dict
Generator expression	(expr for x in it)	lazy iterator (no container built)

Anatomy of a comprehension

```
[ expression  for item in iterable  if condition ]
  ^^^^^^^^^  ^^^^^^^^^^^^^^^^^^^^^  ^^^^^^^^^^^^^
  what to    where values come      optional filter
  store      from (the loop)         (keep item only if True)
```

Read it in English: "give me EXPRESSION for each ITEM in ITERABLE if CONDITION".

List comprehension [Beginner]

1. What it does. Builds a new list by applying an expression to every item of an iterable, optionally keeping only items that pass a test.

2. Why it is useful. Shorter, usually faster than a manual append loop (no repeated method lookup), and reads as one idea: *the list is these values*.

3. Syntax.

```
[expression for item in iterable]
[expression for item in iterable if condition]
```

4. Simple example.

```
squares = [x * x for x in range(6)]
print(squares)          # -> [0, 1, 4, 9, 16, 25]
evens = [x for x in range(10) if x % 2 == 0]
print(evens)            # -> [0, 2, 4, 6, 8]
lengths = [len(w) for w in ["hi", "python", "code"]]
print(lengths)          # -> [2, 6, 4]
```

5. Before vs Pythonic.

```
# BEFORE (works, but clumsy)
squares = []
for x in range(10):
    if x % 2 == 0:
        squares.append(x * x)
```

```
# PYTHONIC
squares = [x * x for x in range(10) if x % 2 == 0]
```

6. Common mistake. Using a comprehension only for its **side effects** (printing, appending elsewhere). It builds a throw-away list. Use a plain for loop when you are not collecting values.

```
# BAD: comprehension for side effects
[print(x) for x in range(3)]    # builds [None, None, None] and throws it away
# GOOD
for x in range(3):
    print(x)
```

7. Interview use. Interviewers watch for this: turning a 4-line loop into one clear line signals fluency. Typical asks - squares of evens, lengths of words, flatten, filter negatives. **Always state complexity: O(n).**

Conditional comprehensions: if at the END vs if/else at the FRONT

This is the most tested comprehension detail. There are **two different places** a condition can go, with different meanings:

Position	Meaning	Form
At the end	Filter: decides whether the item is kept at all.	[x for x in a if x > 0]
At the front (ternary)	Transform: every item is kept, but its value depends on a condition. An else is mandatory.	[x if x > 0 else 0 for x in a]

```
a = [3, -1, 4, -5, 9]
print([x for x in a if x > 0])          # filter    -> [3, 4, 9]      (shorter list)
print([x if x > 0 else 0 for x in a])   # transform -> [3, 0, 4, 0, 9] (same length)
print(["even" if x % 2 == 0 else "odd" for x in range(4)]) # -> ['even','odd','even','odd']

# both together: keep positives, then label them
print(["big" if x > 5 else "small" for x in a if x > 0]) # -> ['small','small','big']

# multiple filter conditions (they act as AND)
print([x for x in range(30) if x % 2 == 0 if x % 3 == 0]) # -> [0, 6, 12, 18, 24]
print([x for x in range(30) if x % 2 == 0 and x % 3 == 0]) # same, clearer
```

Common mistake: writing [x for x in a if x > 0 else 0] - that is a SyntaxError. If you need an else, the condition moves to the front.

Set comprehension [Beginner]

1. What it does. Builds a set: duplicates vanish automatically, and membership tests are average O(1).

2. Why it is useful. One-line "unique values of something": unique lengths, unique first letters, visited-state sets in graph problems.

3. Syntax.

```
{expression for item in iterable}
{expression for item in iterable if condition}
```

4. Simple example.

```
words = ["apple", "bob", "cat", "dog", "eagle"]
print({len(w) for w in words})          # -> {3, 5}
print({x % 3 for x in range(10)})       # -> {0, 1, 2}
print({c.lower() for c in "HeLLo" if c.isalpha()}) # letters, lowercased, unique
```

5. Before vs Pythonic.

```
# BEFORE (works, but clumsy)
unique_lengths = set()
for w in words:
    unique_lengths.add(len(w))
```

```
# PYTHONIC
unique_lengths = {len(w) for w in words}
```

6. Common mistake. {} alone is an **empty dict**, not an empty set. Empty set is set(). Also {x for x in a} and set(a) are the same idea - do not write a comprehension that merely copies (set(a) is simpler).

7. Interview use. "Check if all characters are unique": len(set(s)) == len(s). "Number of distinct values": len({x for x in a}). Sets are also the standard way to make visited in BFS/DFS.

Dictionary comprehension [Intermediate]

1. What it does. Builds a dict from an iterable of key: value pairs.

2. Why it is useful. Fast lookup tables, index maps, inverting a dict, filtering a dict, converting two parallel lists into a mapping.

3. Syntax.

```
{key_expr: value_expr for item in iterable}
{key_expr: value_expr for item in iterable if condition}
```

4. Simple example.

```
print({x: x * x for x in range(4)})           # -> {0: 0, 1: 1, 2: 4, 3: 9}
names = ["A", "B", "C"]; marks = [90, 80, 70]
print({n: m for n, m in zip(names, marks)})   # -> {'A': 90, 'B': 80, 'C': 70}
d = {"a": 1, "b": 2, "c": 3}
print({k: v for k, v in d.items() if v >= 2}) # filter    -> {'b': 2, 'c': 3}
print({v: k for k, v in d.items()})           # invert     -> {1: 'a', 2: 'b', 3: 'c'}
print({ch: i for i, ch in enumerate("abc")})  # index map  -> {'a': 0, 'b': 1, 'c': 2}
```

5. Before vs Pythonic.

```
# BEFORE (works, but clumsy)
result = {}
for i, ch in enumerate("abc"):
    result[ch] = i
```

```
# PYTHONIC
result = {ch: i for i, ch in enumerate("abc")}
```

6. Common mistake. Duplicate keys silently overwrite earlier ones: `{x % 3: x for x in range(7)}` keeps only the last x per key. Also, inverting a dict with duplicate values loses entries.

7. Interview use. "Map each value to its index" (Two Sum setup), "invert a mapping", "filter dict by value". Mention that values must be hashable to become keys and that later duplicates overwrite.

Nested comprehensions [Intermediate to Advanced]

Nested loops inside one comprehension are written in the ****same order as the equivalent nested for loops**** - outer loop first, inner loop second, expression at the very front.

```
# Equivalent loops                                # One comprehension (same left-to-right order)
out = []
for row in matrix:
    for x in row:
        out.append(x)
out = [x for row in matrix for x in row]

matrix = [[1, 2, 3],
           [4, 5, 6],
           [7, 8, 9]]

flat = [x for row in matrix for x in row]          # flatten
print(flat)                                         # -> [1, 2, 3, 4, 5, 6, 7, 8, 9]

T = [[row[i] for row in matrix] for i in range(3)] # transpose (comprehension INSIDE comprehension)
print(T)                                            # -> [[1, 4, 7], [2, 5, 8], [3, 6, 9]]

grid = [[0] * 4 for _ in range(3)]                 # 3x4 grid of independent rows
print(grid)                                         # -> [[0,0,0,0],[0,0,0,0],[0,0,0,0]]

pairs = [(i, j) for i in range(3) for j in range(3) if i < j] # combinations of indices
print(pairs)                                        # -> [(0, 1), (0, 2), (1, 2)]

doubled = [[x * 2 for x in row] for row in matrix] # keep the 2D shape
print(doubled)                                     # -> [[2,4,6],[8,10,12],[14,16,18]]
```

Two different nestings - do not mix them up

`[x for row in m for x in row]` -> **one flat list** (two for clauses in ONE comprehension).

`[x for x in row] for row in m` -> **list of lists** (a comprehension inside the expression part).

Rule of thumb: if you need more than two levels or a condition in each level, write ordinary loops. Readability beats cleverness.

The 2D-grid trap (very common interview question): `[[0] * 3] * 3` makes three references to the **same** inner list. Changing `g[0][0]` changes every row. Fix: `[[0] * 3 for _ in range(3)]`. `_` is the conventional name for a variable you never use.

Advanced comprehension notes

- **Scope:** the loop variable lives only inside the comprehension. `[i for i in range(3)]; print(i)` raises `NameError`.

- **Walrus inside comprehensions:** `[y for x in data if (y := f(x)) > 0]` calls `f` once per item and reuses the result.
- **Memory:** a list comprehension builds the entire list. For huge streams use a **generator expression** `(... for ...)` inside `sum`, `max`, `any`, `all`, `''`.`join`.
- **Speed:** a comprehension is typically faster than the equivalent `for` + `append` loop because the loop runs in optimised bytecode. It is **not** faster than a built-in like `sum(range(n))`.
- **Ordering of results:** list comprehension keeps order; set comprehension does not promise any order; dict comprehension keeps insertion order.
- **When NOT to use one:** more than ~2 clauses, needs `try/except`, needs `break`, or needs to update outside state.

```
data = [3, -1, 2]
# generator expression: no intermediate list, constant memory
total = sum(x * x for x in range(1_000_000))
has_neg = any(x < 0 for x in data)          # stops at the first True (short-circuit)
line = ",".join(str(x) for x in [1, 2, 3]) # str.join needs strings -> convert first
```

2. Built-in Functions

Built-ins are always available - no import. Knowing their **edge cases** is what separates "I have used Python" from "I know Python". They fall into four groups:

Group	Functions
Aggregate / numeric	len, sum, min, max, abs, round
Ordering / iteration	sorted, reversed, enumerate, zip, range
Logic / functional	any, all, map, filter
Introspection / characters	type, isinstance, id, ord, chr

Big idea: lazy vs eager

Eager (result exists immediately): len, sum, min, max, sorted (returns a list), abs, round.

Lazy (returns an iterator/sequence object that produces values on demand): reversed, enumerate, zip, map, filter. range is a lazy *sequence* (indexable, re-iterable, O(1) memory). Lazy iterators can be consumed **only once**.

2.1 Aggregate and numeric functions

len() [Beginner]

1. What it does. Returns the number of items in a container (string, list, tuple, set, dict, range, bytes...). Calls the object's `__len__`.

2. Why it is useful. The most-used built-in: loop bounds, emptiness checks, size comparisons.

3. Syntax.

```
len(container)
```

4. Simple example.

```
print(len("hello"))          # -> 5
print(len([1, 2, 3]))        # -> 3
print(len({"a": 1, "b": 2}))  # -> 2 (number of keys)
print(len(range(10, 0, -2)))  # -> 5
print(len([[1, 2], [3]]))    # -> 2 (top level only)
```

5. Before vs Pythonic.

```
# BEFORE (works, but clumsy)
if len(arr) > 0:
    print("has items")
last_index = len(arr) - 1
```

```
# PYTHONIC
if arr:
    print("has items")
last = arr[-1]
# empty containers are falsy
# no need for len(arr) - 1
```

6. Common mistake. len of a generator/iterator raises TypeError (they have no length). Also len(5) fails - only sized containers work. Remember len("") is 0 and len(" ") is 2.

7. Interview use. O(1) for built-in containers. Use `if not arr:` for emptiness, but use `len(arr) == 0` when the value might be None-vs-empty ambiguous and you want to be explicit. Common bug: `range(len(arr) - 1)` on an empty list gives `range(-1)`, which is empty, not an error.

sum() [Beginner]

1. What it does. Adds up an iterable of numbers, starting from an optional start (default 0).

2. Why it is useful. Totals, counting True values, averages, and prefix-sum building blocks.

3. Syntax.

```
sum(iterable, start=0)
```

4. Simple example.

```
print(sum([1, 2, 3, 4]))          # -> 10
print(sum([]))                   # -> 0
print(sum(x * x for x in range(4))) # -> 14 (generator, no list built)
print(sum([1.5, 2.5], 10))       # -> 14.0 (start=10)
print(sum(1 for x in [4, 7, 9] if x > 5)) # -> 2 (count items matching a test)
print(sum([True, False, True]))   # -> 2 (True == 1)
```

5. Before vs Pythonic.

```
# BEFORE (works, but clumsy)
total = 0
for x in arr:
    total += x
count = 0
for x in arr:
    if x % 2 == 0:
        count += 1
```

```
# PYTHONIC
total = sum(arr)
count = sum(x % 2 == 0 for x in arr) # booleans add up as 0/1
```

6. Common mistake. `sum(list_of_strings)` raises `TypeError` - use `''.join(...).sum(list_of_lists, [])` works but is **$O(n^2)$** (copies each time) - use a flatten comprehension or `itertools.chain`. Also float sums accumulate rounding error (`math.fsum` is exact-ish).

7. Interview use. "Count elements satisfying a condition in one line" = `sum(cond for x in a)`. Mean = `sum(a) / len(a)` (guard against empty). Prefix sums: `itertools.accumulate`.

min() and max() [Beginner to Intermediate]

1. What it does. Return the smallest / largest item of an iterable, or of several positional arguments. Accept `key=` (how to compare) and `default=` (value for empty input).

2. Why it is useful. Extremes, `argmax/argmin` (with `key`), clamping values, comparing strings/tuples.

3. Syntax.

```
min(iterable, *, key=None, default=<no default>)
min(a, b, c, ..., key=None)
max(...) # same signatures
```

4. Simple example.

```
print(max([3, 9, 2]))          # -> 9
print(min(4, 1, 8))           # -> 1
print(max("apple", "pear"))    # -> 'pear' (lexicographic)
print(max(["aa", "b", "cccc"], key=len)) # -> 'cccc'
print(max([], default=-1))     # -> -1 (no crash on empty)
print(max([(1, "z"), (3, "a"), (3, "b")])) # -> (3, 'b') (tuples compare element by element)
x = max(0, min(100, 250))      # clamp into [0, 100] -> 100
```

5. Before vs Pythonic.

```
# BEFORE (works, but clumsy)
best = arr[0]
for x in arr:
    if x > best:
        best = x
# argmax (index of max)
idx = 0
for i in range(len(arr)):
    if arr[i] > arr[idx]:
        idx = i
```

```
# PYTHONIC
best = max(arr)
idx = max(range(len(arr)), key=arr.__getitem__) # or key=lambda i: arr[i]
```

6. Common mistake. `max([])` raises `ValueError`. With **ties**, `max/min` return the **first** encountered item. Comparing incompatible types (`max([1, "a"])`) raises `TypeError`. The result of `max(dict)` is the largest **key**,

not value.

7. Interview use. "Longest word" `max(words, key=len)`. "Key with largest value" `max(d, key=d.get)`. "Second largest" - `sorted(set(a))[-2]` or a single-pass with two variables (say complexity: $O(n \log n)$ vs $O(n)$). Clamp idiom `max(lo, min(hi, x))`.

abs() [Beginner]

1. What it does. Absolute value of a number (magnitude of a complex number too).

2. Why it is useful. Distances, differences, tolerance checks (`abs(a - b) < eps`), Manhattan distance.

3. Syntax.

```
abs(x)
```

4. Simple example.

```
print(abs(-7))          # -> 7
print(abs(3.5))         # -> 3.5
print(abs(3 + 4j))      # -> 5.0 (magnitude)
print(abs(2 - 9))       # -> 7 (distance between 2 and 9)
print(abs(1 - 4) + abs(2 - 6)) # Manhattan distance -> 7
```

5. Before vs Pythonic.

```
# BEFORE (works, but clumsy)
if a > b:
    diff = a - b
else:
    diff = b - a
```

```
# PYTHONIC
diff = abs(a - b)
```

6. Common mistake. Comparing floats with `==` after arithmetic: `0.1 + 0.2 == 0.3` is `False`. Use `abs(a - b) < 1e-9` or `math.isclose`. Also `abs(-0.0)` is `0.0`. `abs` of the minimum int is fine in Python (no overflow, unlike C++).

7. Interview use. Closest-value problems (`min(arr, key=lambda v: abs(v - target))`), Manhattan distance on grids, "absolute difference of adjacent elements". **C++ contrast:** no `INT_MIN` overflow trap.

round() [Beginner to Intermediate]

1. What it does. Rounds a number to `ndigits` decimal places (default: nearest integer, returned as `int`). Uses **round-half-to-even** ("banker's rounding").

2. Why it is useful. Formatting results, computing percentages, reducing float noise.

3. Syntax.

```
round(number)          # -> int
round(number, ndigits) # -> same type as number
round(1234, -2)         # negative ndigits rounds to tens/hundreds...
```

4. Simple example.

```
print(round(3.7))      # -> 4
print(round(3.14159, 2)) # -> 3.14
print(round(1234, -2))  # -> 1200
print(round(2.5), round(3.5), round(0.5), round(1.5)) # -> 2 4 0 2 (ties go to EVEN)
print(round(2.675, 2))  # -> 2.67 (binary float representation of 2.675 is slightly below)
```

5. Before vs Pythonic.

```
# BEFORE (works, but clumsy)
# "school" rounding via floor
import math
r = math.floor(x + 0.5)
```

```
# PYTHONIC
r = round(x)          # banker's rounding (statistical, Python default)
r2 = int(x + 0.5)     # school rounding for non-negative x
```

```
s = f"{x:.2f}" # format to 2 decimals (string) - best for display
```

6. Common mistake. Assuming `round(2.5) == 3`. It is 2. Also `round(x, 2)` returns a float that may still print as 2.6700000000000004-like in other operations; use formatting for display and `decimal.Decimal` for money.

7. Interview use. A favourite trick question: "What is `round(2.5)`?" -> 2. Say **round-half-to-even**. For ceiling division use `-(-a // b)` or `math.ceil(a / b)`; for exact integers prefer `(a + b - 1) // b`.

2.2 Ordering and iteration functions

`sorted()` [Beginner to Advanced]

1. What it does. Returns a **new sorted list** from any iterable. Original untouched. Accepts `key=` and `reverse=`. The sort is **stable** (equal items keep their original relative order).

2. Why it is useful. Ordering anything: numbers, strings, tuples, dict items. Stability lets you sort by several criteria in stages.

3. Syntax.

```
sorted(iterable, *, key=None, reverse=False) # -> list
```

4. Simple example.

```
print(sorted([3, 1, 2])) # -> [1, 2, 3]
print(sorted("banana")) # -> ['a','a','a','b','n','n'] (list of chars)
print(sorted([3, 1, 2], reverse=True)) # -> [3, 2, 1]
print(sorted(["bb", "a", "ccc"], key=len)) # -> ['a', 'bb', 'ccc']
print(sorted({"b": 2, "a": 1})) # -> ['a', 'b'] (keys)
print(sorted({"b": 2, "a": 1}.items())) # -> [('a', 1), ('b', 2)]
```

5. Before vs Pythonic.

```
# BEFORE (works, but clumsy)
copy = arr[:]
copy.sort()
# sort dict by value
items = list(d.items())
items.sort(key=lambda kv: kv[1])
```

```
# PYTHONIC
copy = sorted(arr)
by_value = sorted(d.items(), key=lambda kv: kv[1])
anagram_key = "".join(sorted("listen")) # -> 'eilnst' (all anagrams share this key)
```

6. Common mistake. Writing `arr = arr.sort()` (sets `arr` to `None`, since `.sort()` returns `None`). Also `sorted("hello")` returns a **list**, not a string - re-join with `''.join(...)`. Mixed types (`[1, 'a']`) cannot be sorted in Python 3.

7. Interview use. Timsort: $O(n \log n)$ worst case, $O(n)$ on nearly-sorted data, stable. Anagram grouping, custom orderings, sort-then-two-pointer, interval merging. More on `key=` in section 3.

`reversed()` [Beginner]

1. What it does. Returns a **reverse iterator** over a sequence (list, tuple, string, range...). Does not build a new list and does not modify the original.

2. Why it is useful. Iterate backwards without slicing copies; reverse a string idiomatically.

3. Syntax.

```
reversed(sequence) # -> iterator (needs __reversed__ or a sequence protocol)
```

4. Simple example.

```
print(list(reversed([1, 2, 3]))) # -> [3, 2, 1]
for ch in reversed("abc"):
    print(ch, end=" ") # -> c b a
print("".join(reversed("hello"))) # -> 'olleh'
print(list(reversed(range(5)))) # -> [4, 3, 2, 1, 0]
```

5. Before vs Pythonic.

```
# BEFORE (works, but clumsy)
i = len(arr) - 1
while i >= 0:
    print(arr[i])
    i -= 1
```

```
# PYTHONIC
for x in reversed(arr):
    print(x)
s_rev = s[::-1]          # slicing reverse (returns a new str)
arr.reverse()           # in-place, returns None
```

6. Common mistake. Printing `reversed(x)` shows `<list_reverseiterator ...>`. Also `reversed` does **not** work on sets or plain iterators/generators. And `arr = arr.reverse()` sets `arr` to `None`.

7. Interview use. Palindrome: `s == s[::-1]` ($O(n)$ time and space) or two pointers ($O(1)$ space). `reversed(arr)` is $O(1)$ to create; `arr[::-1]` is $O(n)$ and copies. To reverse *and* get indices: `enumerate(reversed(arr))`.

enumerate() [Beginner]

1. What it does. Pairs each item with its index while iterating, producing (index, item) tuples lazily.

2. Why it is useful. You almost never need `range(len(arr))` in Python: `enumerate` is clearer and less error-prone.

3. Syntax.

```
enumerate(iterable, start=0)
```

4. Simple example.

```
arr = ["a", "b", "c"]
for i, x in enumerate(arr):
    print(i, x)                # -> 0 a / 1 b / 2 c
for i, x in enumerate(arr, start=1):
    print(i, x)                # -> 1 a / 2 b / 3 c
print(list(enumerate("hi")))  # -> [(0, 'h'), (1, 'i')]
pos = {ch: i for i, ch in enumerate("abc")} # value -> index map
```

5. Before vs Pythonic.

```
# BEFORE (works, but clumsy)
for i in range(len(arr)):
    print(i, arr[i])
```

```
# PYTHONIC
for i, x in enumerate(arr):
    print(i, x)
```

6. Common mistake. Unpacking in the wrong order: `for x, i in enumerate(arr)` gives `x=index` and `i=value`. Also `enumerate` yields tuples, so `for pair in enumerate(arr)` gives `pair[0]` (index) and `pair[1]` (value). Modifying a list's length while enumerating it causes skipped elements.

7. Interview use. First index of a value, Two Sum (`for i, x in enumerate(nums)`), `start=1` for human-numbered output. `max(enumerate(a), key=lambda t: t[1])` returns (index, value) of the max.

zip() [Beginner to Intermediate]

1. What it does. Combines several iterables element-by-element into tuples; **stops at the shortest** input.

2. Why it is useful. Parallel iteration, building dicts, transposing matrices, comparing adjacent items, pairing keys and values.

3. Syntax.

```
zip(*iterables, strict=False) # strict=True (3.10+) raises if lengths differ
```

4. Simple example.

```

names = ["A", "B", "C"]; marks = [90, 80, 70]
for n, m in zip(names, marks):
    print(n, m)                                # -> A 90 / B 80 / C 70
print(dict(zip(names, marks)))                  # -> {'A': 90, 'B': 80, 'C': 70}
print(list(zip([1, 2, 3], "ab")))              # -> [(1, 'a'), (2, 'b')] (stops at shortest)
m = [[1, 2, 3], [4, 5, 6]]
print(list(zip(*m)))                           # transpose -> [(1, 4), (2, 5), (3, 6)]
a = [1, 5, 9]
print([y - x for x, y in zip(a, a[1:])])        # adjacent differences -> [4, 4]

```

5. Before vs Pythonic.

```

# BEFORE (works, but clumsy)
for i in range(len(names)):
    print(names[i], marks[i])
transposed = [[row[i] for row in m] for i in range(len(m[0]))]

```

```

# PYTHONIC
for n, mk in zip(names, marks):
    print(n, mk)
transposed = list(zip(*m))

```

6. Common mistake. Silent truncation when lengths differ. Use `strict=True` (3.10+) or `itertools.zip_longest`. Also a zip object is **one-shot**: `z = zip(a, b); list(z); list(z)` gives an empty list the second time.

7. Interview use. "Longest common prefix": for chars in `zip(*strs)`. Compare two lists element-wise, build a dict from two lists, transpose a matrix, iterate over consecutive pairs with `zip(a, a[1:])`.

range() [Beginner]

1. What it does. Produces an immutable **sequence of integers** lazily: `range(stop)`, `range(start, stop)`, `range(start, stop, step)`. `stop` is **excluded**.

2. Why it is useful. Counted loops, index generation, arithmetic progressions - using O(1) memory.

3. Syntax.

```

range(stop)
range(start, stop)
range(start, stop, step)    # step may be negative

```

4. Simple example.

```

print(list(range(5)))        # -> [0, 1, 2, 3, 4]
print(list(range(2, 7)))     # -> [2, 3, 4, 5, 6]
print(list(range(0, 10, 3))) # -> [0, 3, 6, 9]
print(list(range(5, 0, -1))) # -> [5, 4, 3, 2, 1] (stop 0 is excluded)
print(list(range(5, -1, -1))) # -> [5, 4, 3, 2, 1, 0]
r = range(10)
print(r[3], r[-1], 7 in r, len(r)) # -> 3 9 True 10 (indexable, O(1) membership)

```

5. Before vs Pythonic.

```

# BEFORE (works, but clumsy)
i = 0
while i < 10:
    print(i)
    i += 1
for i in range(len(arr) - 1, -1, -1): # backwards, the hard way
    print(arr[i])

```

```

# PYTHONIC
for i in range(10):
    print(i)
for x in reversed(arr):
    print(x)

```

6. Common mistake. Off-by-one: `range(1, 5)` stops at 4. Counting down needs a negative step **and** the stop one beyond the target (`range(n, -1, -1)` to include 0). Floats are not allowed (`range(0, 1, 0.1)` raises `TypeError`). `range(0)` and `range(5, 1)` are simply empty.

7. Interview use. C++ contrast: `for (int i=0;i<n;i++)` becomes `for i in range(n)`. `range` never builds a list in Python 3 (Python 2's lazy version was called `xrange`). Membership `x in range(a, b, s)` is $O(1)$ - a favourite trivia question. A range object supports slicing and `len`.

2.3 Logic and functional functions

`any()` [Beginner]

1. What it does. Returns True if **at least one** item of an iterable is truthy; **short-circuits** on the first truthy item.

2. Why it is useful. Replaces loops that search for "is there one that...". Works with generator expressions without building a list.

3. Syntax.

```
any(iterable)
```

4. Simple example.

```
print(any([0, 0, 3]))          # -> True
print(any([]))                 # -> False
print(any(x % 2 == 0 for x in [1, 3, 5, 8])) # -> True
print(any(c.isdigit() for c in "abc7"))      # -> True
text = "my cat sat"
print(any(word in text for word in ["cat", "dog"])) # -> True
```

5. Before vs Pythonic.

```
# BEFORE (works, but clumsy)
found = False
for x in arr:
    if x % 2 == 0:
        found = True
        break
```

```
# PYTHONIC
found = any(x % 2 == 0 for x in arr)
```

6. Common mistake. Passing a **list** where a generator would do (`any([f(x) for x in a])` computes all `f(x)` first). Also any tests truthiness, not equality: `any([0, "", None])` is False.

7. Interview use. Memory aid: **any = OR over many items**. Typical asks: "contains duplicate", "password has digit/upper/lower", "is any element negative". Empty input -> False.

`all()` [Beginner]

1. What it does. Returns True only if **every** item is truthy; stops at the first falsy item. `all([])` is **True** (vacuous truth).

2. Why it is useful. Validation checks: sorted?, all positive?, all same type?

3. Syntax.

```
all(iterable)
```

4. Simple example.

```
print(all([2, 4, 6]))          # -> True
print(all([]))                 # -> True (nothing violates it)
print(all(x > 0 for x in [3, -1, 5])) # -> False
arr = [1, 2, 2, 5]
print(all(a <= b for a, b in zip(arr, arr[1:]))) # is arr sorted ascending? -> True
print(all(isinstance(x, int) for x in [1, 2, "3"])) # -> False
```

5. Before vs Pythonic.

```
# BEFORE (works, but clumsy)
ok = True
for x in arr:
    if x <= 0:
```

```
ok = False
break
```

```
# PYTHONIC
ok = all(x > 0 for x in arr)
```

6. Common mistake. Forgetting that `all([])` is `True` (a check like "all elements valid" passes on empty input) and `any([])` is `False`. Also confusing `all(x == 0 ...)` with `not any(...)` - they are related by De Morgan: `all(p(x))` equals `not any(not p(x))`.

7. Interview use. Memory aid: **all = AND over many items**. Interview one-liner for "is the list sorted?" is the `zip(arr, arr[1:])` form above. Know De Morgan's law equivalence.

map() [Intermediate]

1. What it does. Applies a function to every item of one or more iterables, lazily, returning an iterator.

2. Why it is useful. Convert types in bulk (the classic `list(map(int, input().split()))`), apply an existing function without writing a loop.

3. Syntax.

```
map(function, iterable, *more_iterables) # -> lazy iterator
```

4. Simple example.

```
print(list(map(int, ["1", "2", "3"])))      # -> [1, 2, 3]
print(list(map(str.upper, ["a", "b"])))     # -> ['A', 'B']
print(list(map(lambda x: x * 2, [1, 2, 3]))) # -> [2, 4, 6]
print(list(map(lambda a, b: a + b, [1, 2], [10, 20]))) # -> [11, 22] (two iterables)
a, b = map(int, "3 5".split())              # unpack directly -> a=3, b=5
arr = list(map(int, input().split()))        # competitive-programming input line
```

5. Before vs Pythonic.

```
# BEFORE (works, but clumsy)
nums = []
for token in "1 2 3".split():
    nums.append(int(token))
```

```
# PYTHONIC
nums = list(map(int, "1 2 3".split()))
nums = [int(t) for t in "1 2 3".split()] # equally Pythonic; many prefer this form
```

6. Common mistake. Printing or reusing a map object: `m = map(int, ["1"]); list(m); list(m)` -> second result is `[]`. Also `len(map(...))` fails. Another trap: in Python 2 `map` returned a list; old tutorials still show that.

7. Interview use. Input parsing in competitive programming. When the function is an existing name (`int`, `str.strip`, `len`) `map` is elegant; when you would need a `lambda`, a comprehension is usually clearer. Both are $O(n)$.

filter() [Intermediate]

1. What it does. Keeps only the items for which a function returns truthy (or, with `None`, only truthy items). Lazy iterator.

2. Why it is useful. Removing unwanted items with an existing predicate function.

3. Syntax.

```
filter(function_or_None, iterable) # -> lazy iterator
```

4. Simple example.

```
print(list(filter(lambda x: x % 2 == 0, [1, 2, 3, 4, 5]))) # -> [2, 4]
print(list(filter(None, [0, 1, "", "a", None, [], [2]]))) # -> [1, 'a', [2]] (drops falsy)
print(list(filter(str.isdigit, ["12", "ab", "7"])))      # -> ['12', '7']
```

5. Before vs Pythonic.

```
# BEFORE (works, but clumsy)
result = []
for x in arr:
    if x % 2 == 0:
        result.append(x)
```

```
# PYTHONIC
result = [x for x in arr if x % 2 == 0]      # preferred for readability
result = list(filter(None, arr))           # good use of filter: drop falsy values
```

6. Common mistake. Assuming `filter(None, [0, 1, 2])` keeps only `None`-removed items - it removes **every falsy** item, including `0` and `""`. If `0` is a valid value, use an explicit condition. And like `map`, the result is one-shot.

7. Interview use. Know that a comprehension `[x for x in a if p(x)]` is equivalent and more common in modern Python. Use `filter` with a named function (`filter(str.isalpha, s)`) or with `None`.

2.4 Introspection and character functions

`type()` [Beginner]

1. What it does. Returns the exact class of an object. With three arguments it dynamically creates a class (advanced, rarely needed).

2. Why it is useful. Debugging, quick sanity checks, printing what a value really is.

3. Syntax.

```
type(obj)
```

4. Simple example.

```
print(type(5))          # -> <class 'int'>
print(type(5.0))        # -> <class 'float'>
print(type("a"))        # -> <class 'str'>
print(type([1]))         # -> <class 'list'>
print(type({}), type(set()), type(())) # dict, set, tuple
print(type(None))       # -> <class 'NoneType'>
print(type(5) == int)   # -> True
print(type(True) == int) # -> False (bool is its own class...)
print(isinstance(True, int)) # -> True (...but a SUBCLASS of int)
```

5. Before vs Pythonic.

```
# BEFORE (works, but clumsy)
if type(x) == int or type(x) == float:
    print("number")
```

```
# PYTHONIC
if isinstance(x, (int, float)):
    print("number")
```

6. Common mistake. Using `type(x) == SomeClass` for checks: it ignores inheritance (`type(True) == int` is `False`). Prefer `isinstance` for checks and use `type` for display/debugging.

7. Interview use. "What does `type(3/2)` return?" -> `float`; `type(3//2)` -> `int`; `type((1))` -> `int` but `type(1,)` -> `tuple` (single-element tuple needs the trailing comma).

`isinstance()` [Beginner to Intermediate]

1. What it does. Checks whether an object is an instance of a class **or of any subclass**; accepts a tuple of classes.

2. Why it is useful. Type validation, handling mixed input (int or list), polymorphic code.

3. Syntax.

```
isinstance(obj, cls)
isinstance(obj, (cls1, cls2, ...))
```

4. Simple example.

```

print(isinstance(3, int))          # -> True
print(isinstance(3, (int, float))) # -> True
print(isinstance("a", str))       # -> True
print(isinstance([1], list))      # -> True
print(isinstance(True, int))      # -> True (bool subclasses int)
print(isinstance(3, bool))        # -> False
def total(x):
    return sum(x) if isinstance(x, list) else x

```

5. Before vs Pythonic.

```

# BEFORE (works, but clumsy)
if type(x) == list or type(x) == tuple:
    ...

```

```

# PYTHONIC
if isinstance(x, (list, tuple)):
    ...

```

6. Common mistake. Forgetting that bool is a subclass of int: `isinstance(True, int)` is True, so a check "is this an integer?" also accepts True/False. Exclude explicitly: `isinstance(x, int)` and not `isinstance(x, bool)`.

7. Interview use. Flatten a nested list recursively: `if isinstance(item, list): recurse else append`. Prefer `isinstance` over `type() ==`. Mention duck typing ("ask forgiveness, not permission") as the alternative philosophy.

id() [Intermediate]

1. What it does. Returns an integer that uniquely identifies an object **during its lifetime** (in CPython, its memory address).

2. Why it is useful. Understanding references: two names, one object? `a is b` is equivalent to `id(a) == id(b)`.

3. Syntax.

```
id(obj)
```

4. Simple example.

```

a = [1, 2]
b = a                # same object
c = a.copy()         # new object
print(id(a) == id(b)) # -> True
print(id(a) == id(c)) # -> False
print(a is b, a is c) # -> True False
print(a == c)         # -> True (equal VALUES, different objects)
x = 256; y = 256
print(x is y)         # -> True (small ints are cached in CPython - implementation detail)

```

5. Before vs Pythonic.

```

# BEFORE (works, but clumsy)
# comparing addresses manually
if id(a) == id(b):
    ...

```

```

# PYTHONIC
if a is b:
    ...
if x is None:                # the ONLY correct way to test for None

```

6. Common mistake. Using `is` to compare numbers/strings. `a is b` on equal large ints or strings may be True or False depending on interning - an implementation detail. **Rule:** `==` compares values, `is` compares identity; use `is` only for None, True, False.

7. Interview use. Classic question: `"a = [1,2]; b = a; b.append(3); print(a)" -> [1, 2, 3]`. Use `id()` to *prove* aliasing during explanation. Never write code that depends on id values.

ord() and chr() [Beginner to Intermediate]

1. What it does. ord(ch) converts a **single character** to its Unicode code point (integer); chr(n) does the reverse. For ASCII letters ord('a') == 97, ord('A') == 65, ord('0') == 48.

2. Why it is useful. Character arithmetic: shifting letters (Caesar cipher), mapping letters to 0-25 indices, counting letters in a fixed-size array, digit values.

3. Syntax.

```
ord(character)    # str of length 1 -> int
chr(code_point)   # int -> str of length 1
```

4. Simple example.

```
print(ord("a"), ord("A"), ord("0"))    # -> 97 65 48
print(chr(97), chr(65), chr(48))       # -> a A 0
print(ord("e") - ord("a"))             # -> 4   (index of 'e' in the alphabet)
print(chr(ord("a") + 2))               # -> 'c'
print(ord(str(7)) - ord("0"))          # -> 7   (digit char -> int, same as int("7"))
def shift(ch, k):                      # Caesar shift for lowercase letters
    return chr((ord(ch) - ord("a") + k) % 26 + ord("a"))
print(shift("y", 3))                  # -> 'b'
counts = [0] * 26
for ch in "hello":
    counts[ord(ch) - ord("a")] += 1
```

5. Before vs Pythonic.

```
# BEFORE (works, but clumsy)
alphabet = "abcdefghijklmnopqrstuvwxyz"
idx = alphabet.index(ch)
nxt = alphabet[(idx + 1) % 26]
```

```
# PYTHONIC
idx = ord(ch) - ord("a")
nxt = chr((idx + 1) % 26 + ord("a"))
```

6. Common mistake. ord("ab") raises TypeError (must be exactly one character). Also forgetting to subtract ord('a') when using letters as array indices, and forgetting % 26 when wrapping around.

7. Interview use. Caesar cipher, anagram check with a 26-slot array, "add strings" / digit arithmetic, str.isalpha alternatives. **C++ contrast:** in C++ 'a' + 1 is an int arithmetic on chars; in Python chars are 1-length strings so you must convert with ord/chr.

3. Pythonic Techniques

"Pythonic" means using the language's own idioms so code is short, readable, and hard to get wrong. Each card below is a small technique you will use daily.

Multiple assignment [Beginner]

1. What it does. Assigns several names in one statement; also chains one value to several names.

2. Why it is useful. Initialises related variables together and makes state updates atomic and readable.

3. Syntax.

```
a, b, c = 1, 2, 3
x = y = 0          # chained: both names refer to the same object
```

4. Simple example.

```
a, b, c = 10, "hi", [1, 2]
print(a, b, c)          # -> 10 hi [1, 2]
lo, hi = 0, len("hello") - 1
prev, cur = 0, 1        # Fibonacci state
for _ in range(5):
    prev, cur = cur, prev + cur
print(cur)              # -> 8
```

5. Before vs Pythonic.

```
# BEFORE (works, but clumsy)
prev = 0
cur = 1
for _ in range(5):
    tmp = prev + cur
    prev = cur
    cur = tmp
```

```
# PYTHONIC
prev, cur = 0, 1
for _ in range(5):
    prev, cur = cur, prev + cur
```

6. Common mistake. Chained assignment with **mutable** values: `a = b = []` makes both names point to the same list. Right side is evaluated **fully first**, then assigned - so `prev, cur = cur, prev + cur` is safe.

7. Interview use. Fibonacci, GCD (`a, b = b, a % b`), two-pointer setup (`l, r = 0, n - 1`), sliding-window init. Tuple packing/unpacking is the mechanism behind all of it.

Swapping variables [Beginner]

1. What it does. Exchanges two values without a temporary variable.

2. Why it is useful. Avoids the temp-variable ritual and works with list elements too.

3. Syntax.

```
a, b = b, a
arr[i], arr[j] = arr[j], arr[i]
```

4. Simple example.

```
a, b = 3, 8
a, b = b, a
print(a, b)          # -> 8 3
arr = [10, 20, 30]
arr[0], arr[2] = arr[2], arr[0]
print(arr)           # -> [30, 20, 10]
```

5. Before vs Pythonic.

```
# BEFORE (works, but clumsy)
tmp = a
a = b
```

```
b = tmp
```

```
# PYTHONIC  
a, b = b, a
```

6. Common mistake. The right side is built first, then targets are assigned **left to right**: `i, arr[i] = arr[i], i` assigns `i` first, so `arr[i]` then uses the **new** `i` (a classic linked-list bug). Keep swaps simple. Also `a = b; b = a` does **not** swap.

7. Interview use. Reverse an array in place with two pointers (`arr[l], arr[r] = arr[r], arr[l]`), bubble sort, partition step of quicksort, permutation generation.

Unpacking [Beginner to Intermediate]

1. What it does. Splits an iterable into separate names according to position. The number of names must match the number of items (unless using *).

2. Why it is useful. Cleaner code than indexing (`pair[0], pair[1]`), and required for readable loops over tuples.

3. Syntax.

```
a, b = iterable  
a, (b, c) = 1, (2, 3)      # nested  
for x, y in list_of_pairs: ...
```

4. Simple example.

```
point = (10, 20)  
x, y = point                # -> x=10, y=20  
name, (m1, m2) = "Ann", (80, 95)  
first, second, third = "abc" # any iterable: chars -> 'a','b','c'  
for i, (a, b) in enumerate([(1, 2), (3, 4)]):  
    print(i, a, b)           # -> 0 1 2 / 1 3 4  
_, second_value = ("ignore", "keep") # _ = "I don't need this"
```

5. Before vs Pythonic.

```
# BEFORE (works, but clumsy)  
point = (10, 20)  
x = point[0]  
y = point[1]
```

```
# PYTHONIC  
x, y = point
```

6. Common mistake. `ValueError: too many values to unpack / not enough values when counts differ`. Unpacking a dict yields its **keys**. Unpacking a string yields characters.

7. Interview use. Parsing `a, b = map(int, input().split())`, iterating for `k, v in d.items()`, returning several values from a function (which really returns one tuple).

Extended unpacking with * [Intermediate]

1. What it does. A starred name in an assignment collects **all remaining items into a list**.

2. Why it is useful. Split "head and tail", "first, middle, last" without slicing arithmetic.

3. Syntax.

```
first, *rest = iterable  
*init, last = iterable  
first, *middle, last = iterable
```

4. Simple example.

```
a, *b = [1, 2, 3, 4]  
print(a, b)                # -> 1 [2, 3, 4]  
*a, b = [1, 2, 3, 4]  
print(a, b)                 # -> [1, 2, 3] 4  
first, *mid, last = [1, 2, 3, 4, 5]  
print(first, mid, last)     # -> 1 [2, 3, 4] 5  
x, *rest = [7]
```

```
print(x, rest)           # -> 7 []      (star part can be EMPTY, and is always a list)
head, *tail = "hello"
print(head, tail)        # -> h ['e', 'l', 'l', 'o']
```

5. Before vs Pythonic.

```
# BEFORE (works, but clumsy)
first = arr[0]
middle = arr[1:-1]
last = arr[-1]
```

```
# PYTHONIC
first, *middle, last = arr
```

6. Common mistake. Only **one** starred name is allowed per assignment. The starred variable is **always a list**, even when unpacking a tuple or string. It fails with an empty iterable if fixed names cannot be filled.

7. Interview use. Recursive list processing ("head + rest"), splitting CSV lines (name, *scores = row), parsing commands (cmd, *args = line.split()).

The * and ** operators [Intermediate]

1. What it does. * and ** have several jobs depending on context: (a) **collect** extra arguments in a function definition (*args, **kwargs); (b) **unpack** an iterable / dict when calling a function; (c) **merge** containers in literals; (d) * alone in a parameter list forces later parameters to be keyword-only.

2. Why it is useful. Flexible function signatures, forwarding arguments, merging lists/dicts, and printing lists without brackets.

3. Syntax.

```
def f(*args, **kwargs): ...      # collect
f(*list_or_tuple, **some_dict)  # unpack at call
[*a, *b]                        {**d1, **d2} # merge in literals
def g(a, *, key=None): ...      # keyword-only after *
```

4. Simple example.

```
def add(*args):                  # args is a TUPLE
    return sum(args)
print(add(1, 2, 3, 4))          # -> 10

def show(**kwargs):              # kwargs is a DICT
    print(kwargs)
show(name="Ann", age=20)         # -> {'name': 'Ann', 'age': 20}

nums = [1, 2, 3]
print(*nums)                    # -> 1 2 3      (unpack into separate arguments)
print(*nums, sep=", ")         # -> 1, 2, 3
d = {"a": 1, "b": 2}
def f(a, b): return a + b
print(f(**d))                   # -> 3          (unpack dict into keyword arguments)
print([*nums, 4, *[5, 6]])      # -> [1, 2, 3, 4, 5, 6]
print(**d, "b": 99, "c": 3)     # -> {'a': 1, 'b': 99, 'c': 3}  (later keys win)
print(max(*nums), min(*nums))   # -> 3 1
```

5. Before vs Pythonic.

```
# BEFORE (works, but clumsy)
print(" ".join(str(x) for x in nums))
merged = a + b
merged_d = d1.copy(); merged_d.update(d2)
```

```
# PYTHONIC
print(*nums)
merged = [*a, *b]
merged_d = {**d1, **d2}          # or d1 | d2 in Python 3.9+
```

6. Common mistake. Mixing order in definitions: it must be `def f(pos, *args, kw=1, **kwargs)`. * vs ** at a call site: `f(*d)` unpacks the dict's **keys** as positional arguments, `f(**d)` unpacks key=value pairs. ** needs string keys.

7. Interview use. "Explain *args vs **kwargs" (tuple vs dict, positional vs keyword). Decorators forward everything with wrapper(*args, **kwargs). zip(*matrix) transposes. print(*arr) for quick output.

C++ contrast

*args looks like C++ variadic templates but is much simpler: it is just a tuple at runtime. * in a call (f(*arr)) is NOT pointer dereference - it means "spread these items as separate arguments".

Ternary expression (conditional expression) [Beginner]

1. What it does. A one-line if/else that **produces a value**: value_if_true if condition else value_if_false.

2. Why it is useful. Choosing between two values inside an assignment, a return, an f-string, or a comprehension.

3. Syntax.

```
result = A if condition else B
```

4. Simple example.

```
x = 5
print("positive" if x > 0 else "non-positive")    # -> positive
sign = 1 if x > 0 else (-1 if x < 0 else 0)      # chained; keep it short
n = None
print(n if n is not None else "default")         # -> default
print(n or "default")                           # -> default (works because None is falsy)
a, b = 4, 9
print(max(a, b) if a != b else "equal")         # -> 9
print(f"{x} is {'even' if x % 2 == 0 else 'odd'}") # inside an f-string
```

5. Before vs Pythonic.

```
# BEFORE (works, but clumsy)
if x > 0:
    result = "positive"
else:
    result = "negative"
```

```
# PYTHONIC
result = "positive" if x > 0 else "negative"
```

6. Common mistake. Writing it in C order: x > 0 ? a : b is a syntax error. The order is **value_if_true first**. Also nested ternaries quickly become unreadable - switch to if/elif/else. And n or default is wrong when a legitimate value is 0 or "" (they are falsy).

7. Interview use. Base cases of recursion, clamping, labelling in comprehensions (["E" if x % 2 == 0 else "O" for x in a]). Know the or shortcut and its falsy-value trap.

lambda [Intermediate]

1. What it does. Creates a small **anonymous function** in one expression: lambda parameters: expression. It returns the expression's value automatically. It can hold only **one expression** (no statements, no assignments).

2. Why it is useful. Perfect as a throw-away key= function for sorted, min, max, heapq, and for short map/filter callbacks.

3. Syntax.

```
lambda x: expression
lambda a, b: a + b
lambda: 42                # no parameters
```

4. Simple example.

```
square = lambda x: x * x
print(square(6))                # -> 36
add = lambda a, b=1: a + b
print(add(5), add(5, 10))       # -> 6 15
pairs = [(1, "z"), (2, "a"), (3, "m")]
```

```
print(sorted(pairs, key=lambda p: p[1]))      # -> [(2, 'a'), (3, 'm'), (1, 'z')]
print(max(pairs, key=lambda p: p[0]))        # -> (3, 'm')
print(list(map(lambda x: x + 1, [1, 2, 3])))  # -> [2, 3, 4]
```

5. Before vs Pythonic.

```
# BEFORE (works, but clumsy)
def second(p):
    return p[1]
pairs.sort(key=second)
```

```
# PYTHONIC
pairs.sort(key=lambda p: p[1])
# or, no lambda needed at all:
from operator import itemgetter
pairs.sort(key=itemgetter(1))
```

6. Common mistake. Late binding in closures: `fs = [lambda: i for i in range(3)]; [f() for f in fs]` gives `[2, 2, 2]`, not `[0, 1, 2]`, because each lambda looks up `i` when called. Fix with a default: `lambda i=i: i`. Also, assigning a lambda to a name (`f = lambda ...`) is discouraged by PEP 8 - use `def`.

7. Interview use. "Sort by the second element", "sort by length then alphabet", "largest by custom rule". Explain that a lambda is just a function object; the late-binding trap appears in intermediate/advanced interviews.

key= (the key function) [Intermediate]

1. What it does. `key` is a function that turns each item into the value used for **comparison**. It is called **once per item**, and the `sort/min/max` compares those keys, not the items themselves. Accepted by `sorted`, `list.sort`, `min`, `max`, `heapq.nlargest/nsmallest`, `itertools.groupby`, `max`.

2. Why it is useful. Sort or pick by any property (length, second field, absolute value, custom score) without writing a comparison function. Faster than old-style `cmp` because keys are computed once (**decorate-sort-undecorate** built in).

3. Syntax.

```
sorted(iterable, key=func)
lst.sort(key=func)
min(iterable, key=func)      max(iterable, key=func)
```

4. Simple example.

```
words = ["banana", "kiwi", "apple", "fig"]
print(sorted(words, key=len))          # -> ['fig', 'kiwi', 'apple', 'banana']
print(sorted(words, key=str.lower))    # case-insensitive alphabetical
print(sorted([-5, 3, -1, 4], key=abs))  # -> [-1, 3, 4, -5]
print(max(words, key=len))             # -> 'banana'
print(min(words, key=lambda w: w[-1])) # word with smallest LAST letter -> 'banana' ('a')
```

5. Before vs Pythonic.

```
# BEFORE (works, but clumsy)
# old approach: manual decorate-sort-undecorate
decorated = [(len(w), w) for w in words]
decorated.sort()
result = [w for _, w in decorated]
```

```
# PYTHONIC
result = sorted(words, key=len)
```

6. Common mistake. Passing `key=len()` (calls the function) instead of `key=len` (passes the function). Also, **key must return comparable values**: mixing `None` and `ints` raises `TypeError`. And `key` is a keyword-only argument: `sorted(words, len)` is an error.

7. Interview use. Every custom-sort question is really a `key=` question. Remember the `argmax` form `max(range(n), key=lambda i: a[i])`.

Custom sorting [Intermediate to Advanced]

Custom order = choose a **key** that captures the rule. Because the sort is **stable** and Python compares tuples element by element, you can express multi-level rules with a tuple key.

```
students = [("Ann", 90), ("Bob", 85), ("Cid", 90), ("Dee", 85), ("Eli", 70)]

# 1. Highest mark first, then name A-Z (tuple key; negate numbers to reverse just one criterion)
print(sorted(students, key=lambda s: (-s[1], s[0])))
# -> [('Ann',90), ('Cid',90), ('Bob',85), ('Dee',85), ('Eli',70)]

# 2. Sort strings by length, then alphabetically
words = ["pear", "fig", "apple", "kiwi", "date"]
print(sorted(words, key=lambda w: (len(w), w)))
# -> ['fig', 'date', 'kiwi', 'pear', 'apple']

# 3. Two-pass stable sort (needed when you cannot negate, e.g. strings)
r = sorted(students, key=lambda s: s[0], reverse=True) # secondary: name Z-A first
r = sorted(r, key=lambda s: s[1]) # primary: mark ascending (stable)
print(r)

# 4. Move zeros to the end but keep others' order (stable + boolean key: False < True)
a = [0, 3, 0, 5, 7]
print(sorted(a, key=lambda x: x == 0)) # -> [3, 5, 7, 0, 0]

# 5. Sort a dict by value descending, ties by key
d = {"x": 2, "y": 5, "z": 2}
print(sorted(d.items(), key=lambda kv: (-kv[1], kv[0]))) # -> [('y',5), ('x',2), ('z',2)]

# 6. Sort by a custom priority list
order = {"high": 0, "mid": 1, "low": 2}
tasks = ["low", "high", "mid", "high"]
print(sorted(tasks, key=order.get)) # -> ['high', 'high', 'mid', 'low']
```

Advanced - a real comparator: if the rule cannot be turned into a key (e.g. "largest number formed by concatenation"), use `functools.cmp_to_key` with a function returning negative / zero / positive.

```
from functools import cmp_to_key
nums = [3, 30, 34, 5, 9]
def cmp(a, b): # put a before b if a+b makes a bigger number than b+a
    return -1 if str(a) + str(b) > str(b) + str(a) else 1
print("".join(map(str, sorted(nums, key=cmp_to_key(cmp))))) # -> 9534330
```

min / max with key [Intermediate]

1. What it does. Selects the item whose **key** is smallest / largest, and returns the **item** (not the key).

2. Why it is useful. Argmax/argmin without a manual loop; "best by rule" problems.

3. Syntax.

```
min(iterable, key=func, default=None)
max(iterable, key=func, default=None)
```

4. Simple example.

```
d = {"a": 3, "b": 9, "c": 5}
print(max(d, key=d.get)) # -> 'b' (key with largest value)
print(max(d.items(), key=lambda kv: kv[1])) # -> ('b', 9)
words = ["hi", "hello", "hey"]
print(max(words, key=len)) # -> 'hello'
arr = [4, 7, 1, 7]
print(max(range(len(arr)), key=lambda i: arr[i])) # -> 1 (first index of max)
print(min(arr, key=lambda v: abs(v - 6))) # -> 7 (closest to 6)
pts = [(1, 2), (5, 1), (0, 0)]
print(min(pts, key=lambda p: p[0] ** 2 + p[1] ** 2)) # -> (0, 0) (closest to origin)
```

5. Before vs Pythonic.

```
# BEFORE (works, but clumsy)
best = None
best_len = -1
for w in words:
```

```
if len(w) > best_len:
    best, best_len = w, len(w)
```

```
# PYTHONIC
best = max(words, key=len)
```

6. Common mistake. Expecting max to return the key value: `max(words, key=len)` returns a **word**, use `len(max(words, key=len))` for the length. Ties return the **first** item. `max(d)` on a dict compares **keys**, not values.

7. Interview use. Most frequent element (`max(freq, key=freq.get)`), closest value to a target, longest word, `argmax` index.

sorted() vs .sort() [Beginner to Intermediate]

1. What it does. `sorted(x)` returns a **new list** and works on **any iterable**. `lst.sort()` sorts the list **in place**, returns `None`, and exists only for lists.

2. Why it is useful. Choose by whether you need to keep the original and what type you have.

3. Syntax.

```
new_list = sorted(iterable, key=None, reverse=False)
lst.sort(key=None, reverse=False)          # returns None
```

4. Simple example.

```
a = [3, 1, 2]
b = sorted(a)
print(a, b)                # -> [3, 1, 2] [1, 2, 3]    (a unchanged)
a.sort()
print(a)                   # -> [1, 2, 3]            (a changed)
print(a.sort())            # -> None
print(sorted((3, 1, 2)))   # -> [1, 2, 3]            (tuple in, LIST out)
print(sorted("cab"))       # -> ['a', 'b', 'c']
# (3, 1, 2).sort()        # AttributeError: tuples have no .sort()
```

5. Before vs Pythonic.

```
# BEFORE (works, but clumsy)
copy = []
for x in a:
    copy.append(x)
copy.sort()
```

```
# PYTHONIC
copy = sorted(a)
```

6. Common mistake. `a = a.sort()` - the classic bug, `a` becomes `None`. Also, `for x in a.sort():` fails (iterating `None`). Both use **Timsort**, $O(n \log n)$, stable, $O(n)$ extra space at worst.

7. Interview use. "Difference between `sort()` and `sorted()`?" -> in-place vs new list, `None` return, iterable vs list-only. `.sort()` saves memory; `sorted()` is safer (no mutation of caller data).

4. Collection Tools

Python's standard library ships specialised containers that fit common problem shapes. Learn to recognise the shape and reach for the tool:

Tool	Import	Shape of problem	Key cost
Counter	from collections import Counter	count occurrences / frequencies	O(n) build, O(1) lookup
defaultdict	from collections import defaultdict	group items, auto-create missing values	O(1) access
deque	from collections import deque	queue, BFS, sliding window, both-end operations	O(1) at both ends
heapq	import heapq	priority queue, top-k, running min	O(log n) push/pop, O(1) peek
bisect	import bisect	binary search on a sorted list, sorted insertion	O(log n) search

Counter [Beginner to Intermediate]

1. What it does. A dict subclass for **counting hashable items**. Keys are items, values are counts. Missing keys return **0** (no KeyError). Built from any iterable in one call.

2. Why it is useful. Replaces the 4-line frequency loop and adds `most_common`, arithmetic between counters, and easy comparisons (anagrams).

3. Syntax.

```
from collections import Counter
c = Counter(iterable)          # or Counter("aabc"), Counter({"a": 2})
c[x]                          # count (0 if absent)
c.most_common(k)              # list of (item, count) highest first
c.update(iterable)            # ADD counts
c.subtract(iterable)          # subtract counts
c.total()                     # sum of counts (3.10+)
c1 + c2, c1 - c2, c1 & c2, c1 | c2
```

4. Simple example.

```
from collections import Counter
c = Counter([1, 2, 2, 3, 3, 3])
print(c)                      # -> Counter({3: 3, 2: 2, 1: 1})
print(c[3], c[99])            # -> 3 0
print(c.most_common(1))       # -> [(3, 3)]
print(c.most_common(2))       # -> [(3, 3), (2, 2)]
s = Counter("mississippi")
print(s)                      # -> Counter({'i': 4, 's': 4, 'p': 2, 'm': 1})
print(Counter("listen") == Counter("silent")) # -> True (anagram test)
print(Counter("aab") + Counter("abc"))        # -> Counter({'a': 3, 'b': 2, 'c': 1})
print(list(c.elements()))                     # -> [1, 2, 2, 3, 3, 3] (repeat each item count times)
```

5. Before vs Pythonic.

```
# BEFORE (works, but clumsy)
freq = {}
for x in arr:
    if x not in freq:
        freq[x] = 0
    freq[x] += 1
```

```
# PYTHONIC
freq = Counter(arr)
```

6. Common mistake. Counter **is not sorted** by iteration order (it keeps insertion order); only `most_common()` sorts. `c.update("abc")` adds counts (unlike `dict.update`, which replaces values). A count can be zero or negative after subtract, and zero-count keys are still present (`len(c)` counts them) - `+c` removes non-positive counts. `most_common` on ties keeps first-seen order.

7. Interview use. Top-k frequent elements, first unique character, anagram check, majority element, ransom note (not `(Counter(note) - Counter(mag))`). Say: build $O(n)$, `most_common(k)` $O(n \log k)$.

defaultdict [Intermediate]

1. What it does. A dict that **creates a default value automatically** when a missing key is accessed, by calling a factory function you supply (int, list, set, lambda: ...).

2. Why it is useful. Removes if key not in d: boilerplate for counting, grouping, and adjacency lists.

3. Syntax.

```
from collections import defaultdict
d = defaultdict(int)          # missing -> 0
d = defaultdict(list)         # missing -> []
d = defaultdict(set)          # missing -> set()
d = defaultdict(lambda: "N/A")
```

4. Simple example.

```
from collections import defaultdict
d = defaultdict(int)
for ch in "banana":
    d[ch] += 1
print(dict(d))                # -> {'b': 1, 'a': 3, 'n': 2}

groups = defaultdict(list)
for w in ["ant", "bee", "art", "bat"]:
    groups[w[0]].append(w)     # group words by first letter
print(dict(groups))           # -> {'a': ['ant', 'art'], 'b': ['bee', 'bat']}

graph = defaultdict(list)     # adjacency list
for u, v in [(1, 2), (1, 3), (2, 3)]:
    graph[u].append(v); graph[v].append(u)
print(dict(graph))            # -> {1: [2, 3], 2: [1, 3], 3: [1, 2]}
```

5. Before vs Pythonic.

```
# BEFORE (works, but clumsy)
groups = {}
for w in words:
    key = w[0]
    if key not in groups:
        groups[key] = []
    groups[key].append(w)
# or with.setdefault
groups.setdefault(key, []).append(w)
```

```
# PYTHONIC
groups = defaultdict(list)
for w in words:
    groups[w[0]].append(w)
```

6. Common mistake. Merely reading a missing key creates it: if `d["zzz"]`: inserts "zzz" with the default. Use `"zzz" in d` or `d.get("zzz")` to test without inserting. Also the factory must be **callable** (`defaultdict(list)`, not `defaultdict([])`), and a default of int gives 0, not None.

7. Interview use. Group anagrams (`defaultdict(list)` with sorted-word key), adjacency lists, counting. Counter is preferable when you *only* count from an iterable; defaultdict is more general.

deque [Intermediate]

1. What it does. A double-ended queue with **$O(1)$ append and pop at both ends**. Pronounced "deck". Optional maxlen makes a fixed-size window that drops old items automatically.

2. Why it is useful. `list.pop(0)` and `list.insert(0, x)` are **$O(n)$** - shifting every element. `deque.popleft()` is $O(1)$. Essential for BFS and sliding windows.

3. Syntax.

```
from collections import deque
q = deque()                  # or deque([1, 2, 3]), deque(maxlen=3)
```

```

q.append(x)          q.appendleft(x)
q.pop()             q.popleft()
q.extend(it)        q.extendleft(it)    # extendleft reverses order!
q.rotate(k)         # rotate right by k (negative = left)
q[0], q[-1]        # peek ends (index in the MIDDLE is O(n))

```

4. Simple example.

```

from collections import deque
q = deque([1, 2, 3])
q.append(4); q.appendleft(0)
print(q)                # -> deque([0, 1, 2, 3, 4])
print(q.popleft(), q.pop()) # -> 0 4
q.rotate(1)
print(q)                # -> deque([3, 1, 2]) (right rotate)
w = deque(maxlen=3)     # keeps only the last 3 items
for x in range(6):
    w.append(x)
print(w)                # -> deque([3, 4, 5], maxlen=3)
q2 = deque()
q2.extendleft([1, 2, 3])
print(q2)               # -> deque([3, 2, 1]) (each item goes to the left in turn)

```

5. Before vs Pythonic.

```

# BEFORE (works, but clumsy)
queue = []
queue.append(1); queue.append(2)
x = queue.pop(0)          # O(n) - slow for big queues

```

```

# PYTHONIC
from collections import deque
queue = deque()
queue.append(1); queue.append(2)
x = queue.popleft()       # O(1)

```

6. Common mistake. Using `list.pop(0)` for BFS on large inputs (TLE). Indexing the **middle** of a deque is $O(n)$. `extendleft` reverses the input order. deque is not sliceable (`q[1:3]` fails); convert with `list(q)`.

7. Interview use. BFS, level-order traversal, sliding-window maximum (monotonic deque), moving averages (`maxlen`), rotating arrays, queue simulation. **Rule:** whenever you write `pop(0)` or `insert(0, ...)` in a loop, switch to deque.

heapq [Intermediate to Advanced]

1. What it does. Python's **priority queue**, implemented as a **binary min-heap stored in a plain list**. The smallest element is always at index 0.

2. Why it is useful. Get the smallest (or largest, with tricks) item repeatedly in $O(\log n)$; top-k problems; merging sorted streams; Dijkstra; scheduling.

3. Syntax.

```

import heapq
heapq.heappush(heap, item)      # O(log n)
heapq.heappop(heap)            # O(log n) -> smallest
heap[0]                        # O(1) peek smallest
heapq.heapify(lst)             # O(n) in-place
heapq.heappushpop(heap, item)  # push then pop (faster than two calls)
heapq.heapreplace(heap, item)  # pop then push
heapq.nlargest(k, it, key=None) # top-k largest
heapq.nsmallest(k, it, key=None) # top-k smallest
heapq.merge(*sorted_iterables) # lazy merge of sorted inputs

```

4. Simple example.

```

import heapq
h = []
for x in [5, 2, 8, 1, 9]:
    heapq.heappush(h, x)
print(h[0])                # -> 1 (smallest, peek)
print(heapq.heappop(h), heapq.heappop(h)) # -> 1 2

```

```

print(heapq.nlargest(2, [5, 2, 8, 1, 9])) # -> [9, 8]
print(heapq.nsmallest(2, [5, 2, 8, 1, 9])) # -> [1, 2]

# max-heap trick: push NEGATIVES
mh = []
for x in [5, 2, 8]:
    heapq.heappush(mh, -x)
print(-heapq.heappop(mh)) # -> 8 (largest)

# (priority, item) tuples
tasks = []
heapq.heappush(tasks, (2, "write"))
heapq.heappush(tasks, (1, "plan"))
heapq.heappush(tasks, (3, "test"))
print(heapq.heappop(tasks)) # -> (1, 'plan')

# heapify a list in O(n)
lst = [9, 4, 7, 1]
heapq.heapify(lst)
print(lst[0]) # -> 1

```

5. Before vs Pythonic.

```

# BEFORE (works, but clumsy)
# repeatedly find the minimum: O(n) each time -> O(n^2) overall
while arr:
    m = min(arr)
    arr.remove(m)
    process(m)

# PYTHONIC
heapq.heapify(arr) # O(n)
while arr:
    process(heapq.heappop(arr)) # O(log n) each -> O(n log n) overall

```

6. Common mistake. (1) Assuming the list is sorted: it is only **heap-ordered** ($h[0]$ is smallest, the rest is not sorted). (2) Expecting a max-heap: it is a **min-heap**; negate numbers. (3) **Tie-break crash:** $(1, \text{obj1})$ vs $(1, \text{obj2})$ compares the objects if priorities tie, raising `TypeError` for non-comparable items - add a counter: $(\text{priority}, \text{count}, \text{item})$. (4) Changing elements of a heap list by hand breaks the invariant.

7. Interview use. Top-k frequent: keep a size- k min-heap: $O(n \log k)$. **K-th largest:** min-heap of size k . Merge k sorted lists. **Dijkstra:** $(\text{distance}, \text{node})$ tuples. Running median: two heaps. Know `nlargest(k, ...)` is $O(n \log k)$ and `sorted(...)[:k]` is $O(n \log n)$.

bisect [Intermediate]

1. What it does. Binary-search helpers for a **sorted list**. `bisect_left(a, x)` returns the index of the first item $\geq x$; `bisect_right(a, x)` (alias `bisect`) returns the index just after the last item $\leq x$ (first item $> x$). `insort` inserts while keeping order.

2. Why it is useful. $O(\log n)$ search and insertion-position finding without writing your own binary search (and its off-by-one bugs).

3. Syntax.

```

import bisect
i = bisect.bisect_left(a, x) # leftmost insertion point
i = bisect.bisect_right(a, x) # rightmost insertion point (= bisect.bisect)
bisect.insort_left(a, x) # insert keeping sorted order
bisect.insort_right(a, x) # (= bisect.insort)
bisect.bisect_left(a, x, lo, hi) # search only a[lo:hi]
bisect.bisect_left(a, x, key=f) # key parameter exists in 3.10+

```

4. Simple example.

```

import bisect
a = [1, 3, 3, 5, 7]
print(bisect.bisect_left(a, 3)) # -> 1 (first index where 3 could go)
print(bisect.bisect_right(a, 3)) # -> 3 (after the last 3)
print(bisect.bisect_left(a, 4)) # -> 3 (where 4 would be inserted)
print(bisect.bisect_left(a, 9)) # -> 5 (end of the list)

```

```

print(bisect.bisect_right(a, 3) - bisect.bisect_left(a, 3))    # -> 2 (count of 3s)
bisect.insort(a, 4)
print(a)                                                       # -> [1, 3, 3, 4, 5, 7]
# membership test using bisect
i = bisect.bisect_left(a, 5)
print(i < len(a) and a[i] == 5)    # -> True

```

5. Before vs Pythonic.

```

# BEFORE (works, but clumsy)
l, r = 0, len(a) - 1
while l <= r:
    mid = (l + r) // 2
    if a[mid] < x:
        l = mid + 1
    else:
        r = mid - 1
# l is the insertion point

```

```

# PYTHONIC
i = bisect.bisect_left(a, x)

```

6. Common mistake. Using bisect on an **unsorted** list (silently wrong). Forgetting that the result is an **insertion index**, not "found/not found": always check `i < len(a)` and `a[i] == x`. `insort` on a list is $O(n)$ because of shifting, despite the $O(\log n)$ search. Mixing up left/right when counting duplicates.

7. Interview use. "Search insert position", count of a value in a sorted array, floor/ceiling of a value, longest increasing subsequence in $O(n \log n)$ (tails array with `bisect_left`), grade-boundary lookup, "first index $\geq$ target".

Choosing the right collection tool

I need to...	Use	Why
count how many times each item appears	Counter	one line, has <code>most_common</code>
group items by a key into lists/sets	<code>defaultdict(list/set)</code>	no key-exists checks
process items first-in-first-out (BFS)	deque	$O(1)$ <code>popleft</code>
keep only the last k items	<code>deque(maxlen=k)</code>	auto-discards old items
always get the smallest item next	heapq	$O(\log n)$ push/pop
get the largest item next	heapq with negated values	min-heap only
top-k items from a stream	heapq size-k heap / <code>nlargest</code>	$O(n \log k)$
locate a position in sorted data	<code>bisect_left</code> / <code>bisect_right</code>	$O(\log n)$ search
stack (LIFO)	plain list (<code>append/pop</code>)	both ends-of-list operations are $O(1)$
fast membership / de-duplicate	set / dict	average $O(1)$

5. Common Coding Patterns

Interview problems are variations on a small set of **patterns**. Each pattern below uses the tools from sections 1-4. Learn to recognise the trigger phrase in the problem statement, then reach for the pattern.

Trigger phrase in the problem	Pattern
"how many times", "most common", "duplicates"	Frequency counting
"anagram", "letters", "characters"	Counting characters (26-slot list or Counter)
"unique", "remove duplicates", "seen before"	Set / dict.fromkeys
"group by", "categorise"	defaultdict(list)
"two numbers that add up to"	Two-sum hash-map pattern
"valid parentheses", "next greater", "undo"	Stack
"level by level", "shortest path (unweighted)", "first come first served"	Queue / BFS
"k largest", "k-th smallest", "always the best next"	Priority queue (heap)

Frequency counting [Beginner]

1. What it does. Count how many times each distinct value appears, producing a value -> count mapping.

2. Why it is useful. The foundation of dozens of problems: majority element, top-k frequent, anagrams, first unique, duplicates. Turns $O(n^2)$ comparisons into $O(n)$.

3. Syntax.

```
freq = Counter(arr)
freq = {}; freq[x] = freq.get(x, 0) + 1
freq = defaultdict(int); freq[x] += 1
```

4. Simple example.

```
from collections import Counter
arr = [4, 1, 4, 2, 4, 1]
freq = Counter(arr)
print(freq)                # -> Counter({4: 3, 1: 2, 2: 1})
print(freq.most_common(1)[0]) # -> (4, 3)  most frequent value and its count
print([x for x, c in freq.items() if c == 1]) # -> [2]  values appearing once
print(max(freq, key=freq.get)) # -> 4
```

5. Before vs Pythonic.

```
# BEFORE (works, but clumsy)
freq = {}
for x in arr:
    if x not in freq:
        freq[x] = 0
    freq[x] += 1
```

```
# PYTHONIC
freq = Counter(arr)
# or without imports:
freq = {}
for x in arr:
    freq[x] = freq.get(x, 0) + 1
```

6. Common mistake. Counting with `list.count` inside a loop: `[arr.count(x) for x in arr]` is **$O(n^2)$** . Also `freq[x] += 1` on a plain dict raises `KeyError` for a new key.

7. Interview use. Say the trade-off aloud: "hash map, $O(n)$ time, $O(k)$ space for k distinct values". Majority element ($> n/2$): Counter or Boyer-Moore voting for $O(1)$ space.

Counting characters [Beginner to Intermediate]

1. What it does. Frequency counting specialised to strings: count each letter/digit/symbol.

2. Why it is useful. Anagram checks, palindrome permutation, first non-repeating character, ransom note, vowels count.

3. Syntax.

```
Counter(s)
counts = [0] * 26; counts[ord(ch) - ord('a')] += 1 # lowercase a-z only
```

4. Simple example.

```
from collections import Counter
s = "abracadabra"
c = Counter(s)
print(c["a"], c["z"]) # -> 5 0
print(Counter("listen") == Counter("silent")) # -> True
print(sum(ch in "aeiou" for ch in "education")) # -> 5 (vowel count)
first_unique = next((ch for ch in s if c[ch] == 1), None)
print(first_unique) # -> 'c'
print(sorted("listen") == sorted("silent")) # -> True (O(n log n) alternative)
# 26-slot array: fastest, only for a-z
cnt = [0] * 26
for ch in "hello":
    cnt[ord(ch) - ord("a")] += 1
print(cnt[ord("l") - ord("a")]) # -> 2
```

5. Before vs Pythonic.

```
# BEFORE (works, but clumsy)
def is_anagram(a, b):
    if len(a) != len(b):
        return False
    for ch in a:
        if a.count(ch) != b.count(ch): # O(n) inside loop -> O(n^2)
            return False
    return True

# PYTHONIC
def is_anagram(a, b):
    return Counter(a) == Counter(b) # O(n)
```

6. Common mistake. Forgetting **case and spaces**: "Listen" vs "silent" need `.lower()`; ignore spaces with `.replace(" ", "")`. Also `s.count(ch)` inside a loop is $O(n^2)$. And `ord(ch) - ord('a')` breaks for uppercase or non-letters (negative or out-of-range index).

7. Interview use. Sliding-window anagram search: maintain a 26-slot count array as the window moves. "Group anagrams" uses `tuple(sorted(word))` or a 26-count tuple as the dict key. Mention Unicode: a Counter works for any character; a 26-slot array only for a-z.

Counting numbers [Beginner to Intermediate]

1. What it does. Frequency counting for integers; sometimes a direct-address array replaces the dict when the range is small.

2. Why it is useful. Duplicates, single number, missing number, counting sort, digit frequency.

3. Syntax.

```
Counter(nums)
count = [0] * (max_value + 1) # counting-array when values are small non-negative ints
```

4. Simple example.

```
from collections import Counter
nums = [3, 1, 3, 2, 1, 3]
c = Counter(nums)
print([x for x, k in c.items() if k > 1]) # duplicates -> [3, 1]
print(max(c, key=c.get)) # mode -> 3
digits = Counter(str(1123581321))
print(digits["1"]) # -> 4 (digit frequency of a number)
```

```
# counting sort for small ranges
count = [0] * 5
for x in [4, 1, 3, 1, 0]:
    count[x] += 1
print([i for i, k in enumerate(count) for _ in range(k)]) # -> [0, 1, 1, 3, 4]
```

5. Before vs Pythonic.

```
# BEFORE (works, but clumsy)
dups = []
for i in range(len(nums)):
    for j in range(i + 1, len(nums)):
        if nums[i] == nums[j] and nums[i] not in dups:
            dups.append(nums[i])
```

```
# PYTHONIC
dups = [x for x, k in Counter(nums).items() if k > 1]
```

6. Common mistake. A counting **array** needs non-negative integers within a known range; with negatives or huge values use a dict/Counter. Also `str(n)` of a negative number contains `'-'`.

7. Interview use. Missing number ($n*(n+1)//2 - \text{sum}(\text{nums})$), single number (XOR), find duplicates in $O(n)$ time. Mention counting sort is $O(n + k)$.

Removing duplicates [Beginner to Intermediate]

1. What it does. Produce a collection in which each value appears once.

2. Why it is useful. Data cleaning and a frequent warm-up problem. The method depends on whether **order matters**.

3. Syntax.

```
list(set(arr))           # fastest, order NOT preserved
list(dict.fromkeys(arr)) # order preserved (first occurrence kept)
sorted(set(arr))         # unique AND sorted
```

4. Simple example.

```
arr = [3, 1, 3, 2, 1]
print(list(set(arr)))      # -> [1, 2, 3] (order not guaranteed)
print(list(dict.fromkeys(arr))) # -> [3, 1, 2] (keeps first-seen order)
print(sorted(set(arr)))    # -> [1, 2, 3]
seen = set(); out = []
for x in arr:
    if x not in seen:
        seen.add(x); out.append(x)
print(out)                 # -> [3, 1, 2]
print(len(arr) != len(set(arr))) # contains duplicate? -> True
# in-place on a SORTED list with two pointers (O(1) extra space)
a = [1, 1, 2, 2, 3]
w = 1
for r in range(1, len(a)):
    if a[r] != a[r - 1]:
        a[w] = a[r]; w += 1
print(a[:w])               # -> [1, 2, 3]
```

5. Before vs Pythonic.

```
# BEFORE (works, but clumsy)
unique = []
for x in arr:
    if x not in unique:          # O(n) search on a list -> O(n^2) total
        unique.append(x)
```

```
# PYTHONIC
unique = list(dict.fromkeys(arr))
```

6. Common mistake. Assuming `list(set(arr))` keeps order. Also, **unhashable** items (lists, dicts) cannot go in a set: convert to tuples first. And `if x not in unique_list` inside a loop is $O(n^2)$.

7. Interview use. "Remove duplicates from a sorted array in place" (two pointers, $O(1)$ space) vs "from an unsorted array" (set/dict, $O(n)$ space). Always ask whether **order** must be kept.

Sorting with conditions [Intermediate]

1. What it does. Sort by a rule that is not simply "smallest first": several criteria, partitioned groups, or a priority list.

2. Why it is useful. Real data almost never sorts by the natural order. Boolean and tuple keys give you conditions in one line.

3. Syntax.

```
sorted(items, key=lambda x: (primary, secondary, ...))
sorted(items, key=lambda x: condition)          # False sorts before True
```

4. Simple example.

```
nums = [5, -2, 8, -7, 0, 3]
print(sorted(nums, key=lambda x: x < 0))          # non-negatives first -> [5, 8, 0, 3, -2, -7]
print(sorted(nums, key=lambda x: (x % 2, x)))      # evens, then odds -> [-2, 0, 8, -7, 3, 5]
print(sorted(nums, key=lambda x: (x < 0, abs(x)))) # positives by size, then negatives by size
words = ["Banana", "apple", "cherry", "Apple"]
print(sorted(words, key=lambda w: (w.lower(), w))) # case-insensitive, ties -> uppercase first
data = [("Ann", 90), ("Bob", 85), ("Cid", 90)]
print(sorted(data, key=lambda t: (-t[1], t[0])))  # marks desc, names asc
```

5. Before vs Pythonic.

```
# BEFORE (works, but clumsy)
evens = [x for x in nums if x % 2 == 0]
odds = [x for x in nums if x % 2 != 0]
evens.sort()
odds.sort()
result = evens + odds
```

```
# PYTHONIC
result = sorted(nums, key=lambda x: (x % 2, x))
```

6. Common mistake. Reversing only one criterion with `reverse=True` reverses **all**. Negate numbers in the tuple key (`-score`) for mixed order, or use two stable passes for strings. Also, remember `x % 2` for negative numbers in Python is 0 or 1 (never -1).

7. Interview use. Custom-order problems: "sort array by parity", "sort by frequency then value" (`key=lambda x: (-freq[x], x)`), "sort colours". Explain stability out loud.

Grouping [Intermediate]

1. What it does. Collect items that share a property into buckets: property -> list of items.

2. Why it is useful. Group anagrams, group by length, group students by grade, build adjacency lists, bucket sort.

3. Syntax.

```
groups = defaultdict(list)
groups[key_of(item)].append(item)
```

4. Simple example.

```
from collections import defaultdict
words = ["eat", "tea", "tan", "ate", "nat", "bat"]
groups = defaultdict(list)
for w in words:
    groups["".join(sorted(w))].append(w)
print(list(groups.values())) # -> [['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]

by_len = defaultdict(list)
for w in words:
    by_len[len(w)].append(w)
print(dict(by_len))          # -> {3: ['eat', 'tea', 'tan', 'ate', 'nat', 'bat']}

from itertools import groupby # groupby needs SORTED input, groups only ADJACENT equals
for k, g in groupby(sorted(words, key=lambda w: w[0]), key=lambda w: w[0]):
    print(k, list(g))         # -> a ['ate'] / b ['bat'] / e ['eat'] / ...
```

5. Before vs Pythonic.

```
# BEFORE (works, but clumsy)
groups = {}
for w in words:
    k = "".join(sorted(w))
    if k in groups:
        groups[k].append(w)
    else:
        groups[k] = [w]
```

```
# PYTHONIC
groups = defaultdict(list)
for w in words:
    groups["".join(sorted(w))].append(w)
```

6. Common mistake. A **list cannot be a dict key**: use `tuple(sorted(w))` or a string. `itertools.groupby` without sorting first groups only **consecutive** equal keys - a very common surprise. Also, a `groupby` group is a one-shot iterator: convert with `list(g)` immediately.

7. Interview use. "Group Anagrams" is asked constantly. Key choices: sorted string ($O(k \log k)$) or 26-count tuple ($O(k)$). Mention time $O(n * k \log k)$.

Finding maximum / minimum [Beginner to Intermediate]

1. What it does. Locate the largest or smallest value, its index, the k-th largest, or the max under a rule.

2. Why it is useful. Appears everywhere: best score, longest word, peak, second maximum, running max.

3. Syntax.

```
max(arr); min(arr)
max(range(len(arr)), key=arr.__getitem__)    # index of max
heapq.nlargest(k, arr)[-1]                   # k-th largest
```

4. Simple example.

```
arr = [4, 9, 2, 9, 5]
print(max(arr), min(arr))                    # -> 9 2
print(arr.index(max(arr)))                   # -> 1 (first index of the max)
print(sorted(set(arr))[-2])                  # second largest distinct -> 5
import heapq
print(heapq.nlargest(2, arr))                 # -> [9, 9]
# single pass, O(1) space: max and second max
first = second = float("-inf")
for x in arr:
    if x > first:
        first, second = x, first
    elif first > x > second:
        second = x
print(first, second)                         # -> 9 5
# running (prefix) maximum
from itertools import accumulate
print(list(accumulate(arr, max)))            # -> [4, 9, 9, 9, 9]
```

5. Before vs Pythonic.

```
# BEFORE (works, but clumsy)
biggest = arr[0]
for x in arr[1:]:
    if x > biggest:
        biggest = x
```

```
# PYTHONIC
biggest = max(arr)
```

6. Common mistake. Initialising with 0 or -1 as the starting max (wrong for all-negative data). Use `arr[0]` or `float('-inf')`. `max([])` raises `ValueError`. Second largest with duplicates: decide whether "distinct" is required.

7. Interview use. "Second largest" in $O(n)$, k -th largest with a heap ($O(n \log k)$), maximum subarray (Kadane): `cur = max(x, cur + x)`. Use `float('inf')` / `float('-inf')` as sentinels.

Two-sum pattern [Intermediate]

1. What it does. For each number, compute the **complement** (`target - x`) and check whether you have already seen it, using a hash map from value to index.

2. Why it is useful. Turns the $O(n^2)$ double loop into **$O(n)$** time. The same "what do I need? have I seen it?" idea solves pair-difference, subarray-sum-equals- k , and many counting problems.

3. Syntax.

```
seen = {} # value -> index
for i, x in enumerate(nums):
    need = target - x
    if need in seen:
        return seen[need], i
    seen[x] = i
```

4. Simple example.

```
nums, target = [2, 7, 11, 15], 9
seen = {}
for i, x in enumerate(nums):
    need = target - x
    if need in seen:
        print(seen[need], i) # -> 0 1
        break
    seen[x] = i
# Sorted array -> two pointers, O(1) extra space
a = [1, 3, 4, 6, 8]; l, r = 0, len(a) - 1
while l < r:
    s = a[l] + a[r]
    if s == 10: print(l, r); break # -> 2 3
    elif s < 10: l += 1
    else: r -= 1
```

5. Before vs Pythonic.

```
# BEFORE (works, but clumsy)
for i in range(len(nums)):
    for j in range(i + 1, len(nums)):
        if nums[i] + nums[j] == target:
            print(i, j)
```

```
# PYTHONIC
seen = {}
for i, x in enumerate(nums):
    if target - x in seen:
        print(seen[target - x], i)
    seen[x] = i
```

6. Common mistake. Insert after checking, not before: otherwise an element can pair with itself ([3], target 6 would wrongly succeed). Also, a plain set can tell you *whether* a pair exists but cannot return **indices** - store value -> index in a dict. Duplicates: [3, 3] with target 6 works because the first 3 is stored before the second is checked.

7. Interview use. The #1 hash-map interview problem. State: $O(n)$ time, $O(n)$ space; the sorted two-pointer alternative is $O(n \log n)$ time (sorting) and $O(1)$ space but loses original indices. Extensions: 3Sum (sort + two pointers), count pairs with difference k , subarray sum = k with prefix sums + Counter.

Stack pattern (LIFO) [Intermediate]

1. What it does. A stack is a Last-In-First-Out structure. In Python use a **list**: append to push, pop to pop, `stack[-1]` to peek. All $O(1)$ amortised.

2. Why it is useful. Matching brackets, undo history, evaluating expressions, DFS (iterative), "next greater element" (monotonic stack), reversing.

3. Syntax.

```
stack = []
stack.append(x)      # push
top = stack[-1]      # peek (check `if stack:` first)
x = stack.pop()      # pop
if not stack: ...    # empty
```

4. Simple example.

```
def is_valid(s):
    pairs = {"(": ")", "[": "]", "{": "}"
    stack = []
    for ch in s:
        if ch in pairs:
            # closing bracket
            if not stack or stack[-1] != pairs[ch]:
                return False
            stack.pop()
        else:
            # opening bracket
            stack.append(ch)
    return not stack

print(is_valid("({[]})"), is_valid("()"), is_valid("(")) # -> True False False

# monotonic stack: next greater element to the right
def next_greater(a):
    res, st = [-1] * len(a), []
    for i, x in enumerate(a):
        while st and a[st[-1]] < x:
            res[st.pop()] = x
        st.append(i)
    return res
print(next_greater([2, 1, 5, 3, 4])) # -> [5, 5, -1, 4, -1]
```

5. Before vs Pythonic.

```
# BEFORE (works, but clumsy)
# manual "stack" with index bookkeeping
stack = [None] * 100
top = -1
top += 1; stack[top] = 5
val = stack[top]; top -= 1
```

```
# PYTHONIC
stack = []
stack.append(5)
val = stack.pop()
```

6. Common mistake. `stack.pop()` or `stack[-1]` on an **empty** list raises `IndexError` - guard with `if stack:`. Do not use `pop(0)` (that is queue behaviour and $O(n)$). Forgetting the final `return not stack` in bracket matching (unclosed brackets).

7. Interview use. Valid parentheses, min stack, daily temperatures, largest rectangle in histogram, decode string, evaluate reverse Polish notation, iterative DFS. Complexity: each element pushed and popped at most once -> **$O(n)$** .

Queue pattern (FIFO) [Intermediate]

1. What it does. A queue is First-In-First-Out. In Python use `**collections.deque**`: `append` to enqueue, `popleft` to dequeue. Both $O(1)$.

2. Why it is useful. BFS (shortest path in an unweighted graph or grid), level-order tree traversal, task scheduling, sliding window.

3. Syntax.

```
from collections import deque
q = deque([start])
q.append(x)          # enqueue
x = q.popleft()      # dequeue
while q: ...         # loop until empty
```

4. Simple example.

```
from collections import deque
def bfs(graph, start):
    seen = {start}
    q = deque([start])
    order = []
    while q:
        node = q.popleft()
        order.append(node)
        for nxt in graph[node]:
            if nxt not in seen:
                seen.add(nxt)          # mark when ENQUEUED, not when dequeued
                q.append(nxt)
    return order

g = {1: [2, 3], 2: [4], 3: [4], 4: []}
print(bfs(g, 1))                    # -> [1, 2, 3, 4]

# level-by-level: freeze the queue size at the start of each level
def levels(g, s):
    q, seen, out = deque([s]), {s}, []
    while q:
        out.append([q.popleft() for _ in range(len(q))])  # len(q) evaluated once here
        for lvl in out[-1]:
            for nxt in g[lvl]:
                if nxt not in seen:
                    seen.add(nxt); q.append(nxt)
    return out
print(levels(g, 1))                # -> [[1], [2, 3], [4]]
```

5. Before vs Pythonic.

```
# BEFORE (works, but clumsy)
queue = [start]
while queue:
    node = queue.pop(0)          # O(n) each time!
    ...
```

```
# PYTHONIC
queue = deque([start])
while queue:
    node = queue.popleft()      # O(1)
```

6. Common mistake. Using `list.pop(0)` (slow). Marking nodes as visited only when **dequeued** lets the same node be enqueued many times (blow-up). Modifying `len(q)` mid-level loop; capture it first: `for _ in range(len(q))`.

7. Interview use. BFS for shortest path in a maze/grid, binary-tree level order, word ladder, rotting oranges, moving average with `deque(maxlen=k)`. BFS complexity $O(V + E)$.

Priority queue pattern (heap) [Intermediate to Advanced]

1. What it does. A priority queue always hands back the **best** element next (smallest priority number by default in `heapq`).

2. Why it is useful. Top-k, k-th largest, merging sorted lists, task scheduling, Dijkstra, meeting-room problems.

3. Syntax.

```
import heapq
h = []
heapq.heappush(h, (priority, item))
priority, item = heapq.heappop(h)
# max-heap: push (-priority, item)
```

4. Simple example.

```
import heapq
# k-th largest with a size-k MIN-heap: O(n log k)
def kth_largest(nums, k):
```

```

h = []
for x in nums:
    heapq.heappush(h, x)
    if len(h) > k:
        heapq.heappop(h)          # drop the smallest; the heap keeps the k largest
return h[0]
print(kth_largest([3, 2, 1, 5, 6, 4], 2))    # -> 5

# top-k frequent elements
from collections import Counter
def top_k_frequent(nums, k):
    return [x for x, _ in Counter(nums).most_common(k)]    # uses a heap internally
print(top_k_frequent([1, 1, 1, 2, 2, 3], 2))    # -> [1, 2]

# Dijkstra core (weighted graph)
def dijkstra(graph, src):
    dist = {src: 0}
    h = [(0, src)]
    while h:
        d, u = heapq.heappop(h)
        if d > dist.get(u, float("inf")):
            continue          # stale entry, skip
        for v, w in graph[u]:
            nd = d + w
            if nd < dist.get(v, float("inf")):
                dist[v] = nd
                heapq.heappush(h, (nd, v))
    return dist
g = {"A": [("B", 1), ("C", 4)], "B": [("C", 2)], "C": []}
print(dijkstra(g, "A"))      # -> {'A': 0, 'B': 1, 'C': 3}

```

5. Before vs Pythonic.

```

# BEFORE (works, but clumsy)
arr.sort()
smallest_k = arr[:k]          # O(n log n) even when k is tiny

# PYTHONIC
smallest_k = heapq.nsmallest(k, arr)    # O(n log k)

```

6. Common mistake. Forgetting that `heapq` is a **min-heap** when you need the largest (negate or store `-x`). Tuples (priority, item) compare item on ties - non-comparable items crash; add a tie-breaking counter. Popping from an empty heap raises `IndexError`.

7. Interview use. "K-th largest", "merge k sorted lists", "top-k frequent", "task scheduler", "meeting rooms II", "running median" (two heaps), Dijkstra. Complexity: push/pop $O(\log n)$, heapify $O(n)$, peek $O(1)$.

6. Python Shortcuts Cheat Sheet

6.1 Comprehensions and built-ins

Goal	Shortcut	Note
squares of evens	<code>[x*x for x in a if x % 2 == 0]</code>	filter at the END
label each item	<code>["E" if x % 2 == 0 else "O" for x in a]</code>	ternary at the FRONT
unique lengths	<code>{len(w) for w in words}</code>	set comprehension
value -> index map	<code>{v: i for i, v in enumerate(a)}</code>	dict comprehension
flatten 2D	<code>[x for row in m for x in row]</code>	outer loop first
make n x m grid	<code>[[0]*m for _ in range(n)]</code>	never <code>[[0]*m]*n</code>
count matches	<code>sum(cond for x in a)</code>	True adds as 1
index + value	<code>for i, x in enumerate(a, 1)</code>	start=1 optional
parallel loop	<code>for a, b in zip(A, B)</code>	stops at shortest
transpose	<code>list(zip(*m))</code>	tuples; wrap rows in list
adjacent pairs	<code>zip(a, a[1:])</code>	sorted check, diffs
any / all	<code>any(p(x) for x in a) / all(...)</code>	generator, short-circuits
is sorted?	<code>all(a[i] <= a[i+1] for i in range(len(a)-1))</code>	or the zip form
read ints	<code>list(map(int, input().split()))</code>	one line of input
reverse	<code>a[::-1] / reversed(a) / a.reverse()</code>	copy / iterator / in place
clamp	<code>max(lo, min(hi, x))</code>	
char <-> code	<code>ord('a')=97 / chr(97)='a'</code>	<code>ord(c) - ord('a') = 0..25</code>

6.2 Assignment, unpacking, sorting

Goal	Shortcut
swap	<code>a, b = b, a</code>
head / tail	<code>first, *rest = a</code>
first / middle / last	<code>first, *mid, last = a</code>
merge lists	<code>[*a, *b]</code>
merge dicts	<code>{**d1, **d2}</code> (or <code>d1 d2, 3.9+</code>)
print without brackets	<code>print(*a, sep=' ')</code>
ternary	<code>x if cond else y</code>
default for None	<code>val if val is not None else default</code>
sort by length	<code>sorted(w, key=len)</code>
descending	<code>sorted(a, reverse=True)</code>
multi-key (mixed order)	<code>sorted(a, key=lambda t: (-t[1], t[0]))</code>
sort dict by value	<code>sorted(d.items(), key=lambda kv: kv[1])</code>
key of largest value	<code>max(d, key=d.get)</code>
index of max	<code>max(range(len(a)), key=a.__getitem__)</code>
anagram key	<code>"".join(sorted(word))</code>
in-place vs new	<code>a.sort()</code> -> None / <code>sorted(a)</code> -> new list

6.3 Collections module and friends

Tool	Cheat line	Cost
Counter	<code>c = Counter(a); c[x]; c.most_common(k); c1 == c2</code>	$O(n)$ build
defaultdict	<code>d = defaultdict(list); d[k].append(v)</code>	$O(1)$
deque	<code>q = deque(); q.append(x); q.popleft()</code>	$O(1)$ both ends
deque window	<code>deque(maxlen=k)</code>	auto-drops oldest
heap push/pop	<code>heappush(h, x); heappop(h); h[0]</code>	$O(\log n)$ / $O(1)$
heapify	<code>heapify(lst)</code>	$O(n)$
max-heap	push <code>-x</code> , negate on pop	
top-k	<code>nlargest(k, a); nsmallest(k, a)</code>	$O(n \log k)$
bisect	<code>bisect_left(a, x); bisect_right(a, x); insert(a, x)</code>	$O(\log n)$ search
count of x in sorted	<code>bisect_right(a, x) - bisect_left(a, x)</code>	$O(\log n)$
stack	<code>s.append(x); s.pop(); s[-1]</code>	$O(1)$
unique in order	<code>list(dict.fromkeys(a))</code>	$O(n)$

6.4 Complexity quick table

Operation	list	set / dict	deque	heap (list)
access by index	$O(1)$	-	$O(n)$ middle	$O(1)$
<code>x in</code>	$O(n)$	$O(1)$ avg	$O(n)$	$O(n)$
append / add	$O(1)$ amortised	$O(1)$ avg	$O(1)$	$O(\log n)$
pop last	$O(1)$	-	$O(1)$	-
pop first / smallest	$O(n)$	-	$O(1)$	$O(\log n)$
insert at front	$O(n)$	-	$O(1)$	-
sort	$O(n \log n)$	-	-	heapify $O(n)$
min / max	$O(n)$	$O(n)$	$O(n)$	min: $O(1)$

6.5 Ten lines to memorise

<code>freq = Counter(arr)</code>	<code># count everything</code>
<code>uniq = list(dict.fromkeys(arr))</code>	<code># dedupe, keep order</code>
<code>groups = defaultdict(list); groups[key].append(x)</code>	<code># group by key</code>
<code>q = deque([start]); node = q.popleft()</code>	<code># $O(1)$ queue</code>
<code>heappush(h, (priority, item)); heappop(h)</code>	<code># priority queue</code>
<code>i = bisect_left(a, x)</code>	<code># position in sorted list</code>
<code>seen = {}; need = target - x; need in seen</code>	<code># two-sum lookup</code>
<code>arr.sort(key=lambda t: (-t[1], t[0]))</code>	<code># multi-key sort</code>
<code>first, *rest = arr; a, b = b, a</code>	<code># unpack and swap</code>
<code>ok = all(x > 0 for x in arr); any(x < 0 for x in arr)</code>	<code># quick checks</code>

7. Practice Set

Attempt each question **before** reading section 8. For output questions, write your prediction on paper first. Difficulty grows from top to bottom within each group.

7.1 Forty Conceptual Questions

- Q1.** What is a list comprehension, and why is it preferred over a for + append loop?
- Q2.** In a comprehension, what is the difference between if at the end and x if c else y at the front?
- Q3.** Why is `[[0] * 3] * 3` a bug, and how do you fix it?
- Q4.** Does the loop variable of a comprehension leak into the surrounding scope in Python 3?
- Q5.** When should you use a generator expression instead of a list comprehension?
- Q6.** What does `{}` create? How do you create an empty set?
- Q7.** What happens with duplicate keys in a dict comprehension?
- Q8.** Why can you not iterate twice over the result of `map`, `filter`, `zip`, or `enumerate`?
- Q9.** Difference between `sorted()` and `list.sort()`?
- Q10.** What does it mean that Python's sort is stable, and how do you use it?
- Q11.** How many times is the key function called per element, and what does `max(items, key=f)` return?
- Q12.** Why is `round(2.5)` equal to 2?
- Q13.** What do `any([])` and `all([])` return, and why?
- Q14.** What happens with `max([])` and how do you avoid it?
- Q15.** Why does `isinstance(True, int)` return True?
- Q16.** Explain `is` versus `==`. When is `is` correct?
- Q17.** How do `ord()` and `chr()` help in string problems?
- Q18.** What does `zip` do with iterables of different lengths? How can you change that?
- Q19.** Why prefer `enumerate` to `range(len(a))`?
- Q20.** What is the difference between `*args` and `**kwargs`?
- Q21.** What does the starred name do in `first, *rest = items`?
- Q22.** Why does `a, b = b, a` swap correctly without a temp variable?
- Q23.** What is the syntax of the ternary expression, and when is `x or default` a bad substitute?
- Q24.** What are the limits of `lambda`, and what is the late-binding trap?
- Q25.** What is `Counter`, and how does it differ from a normal dict?
- Q26.** What is the danger of reading a missing key of a `defaultdict`?
- Q27.** When should you choose `defaultdict` over `Counter`, and vice versa?
- Q28.** Why is `deque` better than a list for a queue?
- Q29.** Is `heapq` a min-heap or max-heap? How do you get max-heap behaviour?
- Q30.** Is a heap list sorted? What do you know about `h[1]` and `h[2]`?
- Q31.** What goes wrong with `heappush(h, (priority, obj))` and how do you fix it?
- Q32.** Compare `heapify(lst)` with pushing `n` items one at a time.
- Q33.** When is `heapq.nlargest(k, a)` better than `sorted(a, reverse=True)[:k]`?
- Q34.** What is the difference between `bisect_left` and `bisect_right`? What must be true of the list?
- Q35.** Is `bisect.insort` $O(\log n)$? Explain.
- Q36.** How do you remove duplicates while preserving order, and why is `list(set(a))` not enough?
- Q37.** Why are dict and set lookups $O(1)$ on average, and what does "hashable" mean?
- Q38.** Explain why Two Sum stores the value in the dict **after** checking for the complement.

Q39. Give three problems solved by a stack and state why a list is enough as a stack in Python.

Q40. In BFS with a queue, why mark nodes visited when they are enqueued instead of when dequeued?

7.2 Forty Output Questions

What does each program print? (Assume Python 3.12. If it raises an error, say which.)

01.

```
print([x for x in range(10) if x % 3 == 0])
```

02.

```
print([x if x % 2 else -x for x in range(5)])
```

03.

```
g = [[0] * 2] * 2
g[0][0] = 1
print(g)
```

04.

```
g = [[0] * 2 for _ in range(2)]
g[0][0] = 1
print(g)
```

05.

```
print({x % 3: x for x in range(7)})
```

06.

```
print(type({}), type({1}), type({1: 2}))
```

07.

```
i = 100
result = [i for i in range(3)]
print(i)
```

08.

```
print(sum(x for x in range(5) if x % 2))
```

09.

```
m = map(int, ["1", "2"])
print(list(m))
print(list(m))
```

010.

```
print(list(zip([1, 2, 3], "ab")))
print(list(zip(*[[1, 2], [3, 4]])))
```

011.

```
print(round(2.5), round(3.5), round(-0.5), round(1.25, 1))
```

012.

```
print(any([]), all([]), any([0, ""]), all([1, "a"]))
```

013.

```
print(max("apple", "banana"), max(3, 10, 7), min([], default="none"))
```

014.

```
print(max([3, 1, 2], key=lambda x: -x))
```

O15.

```
a = [3, 1, 2]
b = a.sort()
print(a, b)
```

O16.

```
print(sorted("banana"))
```

O17.

```
print(sorted([3, -1, 2, -5], key=abs))
```

O18.

```
words = ["bb", "a", "cc", "d"]
print(sorted(words, key=len))
```

O19.

```
print(list(enumerate("ab", start=5)))
```

O20.

```
print(list(range(10, 0, -3)), len(range(2, 20, 5)), 9 in range(0, 20, 3))
```

O21.

```
print(ord("a") + 1, chr(ord("a") + 1), ord("Z") - ord("A"))
```

O22.

```
a, *b, c = range(5)
print(a, b, c)
```

O23.

```
a, *b = "hi"
print(a, b)
```

O24.

```
def f(*args, **kw):
    return args, kw

print(f(1, 2, x=3))
```

O25.

```
print(["ab", *[1, 2]], {**{"a": 1}, **{"a": 2, "b": 3}})
```

O26.

```
x = [1, 2]
y = x
y += [3]
y = y + [4]
print(x, y)
```

O27.

```
print(5 if 0 else 6, 0 or "d", "" or [] or "z", 3 and 4)
```

O28.

```
fs = [lambda: i for i in range(3)]
print([f() for f in fs])
```

O29.

```
fs = [lambda i=i: i for i in range(3)]
print([f() for f in fs])
```

O30.

```
from collections import Counter
c = Counter("aabbbc")
print(c.most_common(2), c["z"], len(c))
```

O31.

```
from collections import Counter
c = Counter("abc")
c.subtract("aab")
print(c)
```

O32.

```
from collections import defaultdict
d = defaultdict(list)
d["x"]
print(len(d), "y" in d, d.get("y"))
```

O33.

```
from collections import deque
q = deque([1, 2, 3])
q.rotate(1)
q.appendleft(0)
print(q, q.pop())
```

O34.

```
from collections import deque
q = deque(maxlen=2)
for i in range(5):
    q.append(i)
print(q)
```

O35.

```
import heapq
h = [5, 1, 4]
heapq.heapify(h)
print(h[0], heapq.heappop(h), h[0])
```

O36.

```
import heapq
h = []
for x in [3, 1, 2]:
    heapq.heappush(h, -x)
print(-h[0])
```

O37.

```
import heapq
print(heapq.nlargest(2, [4, 1, 9, 9, 3]), heapq.nsmallest(2, "hello"))
```

O38.

```
import bisect
a = [1, 3, 3, 5]
print(bisect.bisect_left(a, 3), bisect.bisect_right(a, 3), bisect.bisect_left(a, 4), bisect.bisect_left(a, 0))
```

O39.

```
a = [1, 2]
b = [1, 2]
print(a == b, a is b, a is a[:], isinstance(True, int))
```

O40.

```
print(-7 // 2, -7 % 2, 7 // -2, divmod(-7, 2))
```

7.3 Forty Coding Problems

Write a function for each problem, then state its time and space complexity. Difficulty is marked Easy / Medium / Hard.

C1. Squares of evens [Easy]

Return a list of the squares of the even numbers in a, keeping order. Example: [1, 2, 3, 4] -> [4, 16].

C2. Flatten a matrix [Easy]

Flatten a list of lists into one list, row by row. Example: [[1, 2], [3], []] -> [1, 2, 3].

C3. Transpose [Easy]

Transpose a rectangular matrix. Example: [[1,2,3],[4,5,6]] -> [[1,4],[2,5],[3,6]].

C4. Invert a dictionary with repeated values [Easy]

Given a dict, return a dict mapping each value to the **list of keys** that had it (in original order).

Example: {'a':1, 'b':2, 'c':1} -> {1:['a', 'c'], 2:['b']}.

C5. Count vowels [Easy]

Count the vowels (aeiou, case-insensitive) in a string. Example: 'Education' -> 5.

C6. Reverse the words [Easy]

Reverse the order of words in a sentence, collapsing extra spaces. Example: ' hello world ' -> 'world hello'.

C7. Palindrome (alphanumeric only) [Easy]

Return True if the string reads the same forwards and backwards, ignoring case and non-alphanumeric characters. Example: 'A man, a plan, a canal: Panama' -> True.

C8. Second largest distinct value [Easy]

Return the second largest **distinct** value, or None if it does not exist. Example: [5, 5, 3] -> 3; [7] -> None. Do it in one pass.

C9. Move zeros to the end [Easy]

Return a list with all zeros moved to the end, other elements keeping their order. Example:

[0,1,0,3,12] -> [1,3,12,0,0].

C10. Is the list sorted? [Easy]

Return True if the list is in non-decreasing order. Example: [1,2,2,5] -> True.

C11. Remove duplicates, keep order [Easy]

Remove duplicates from a list of hashable items keeping the first occurrence. Example: [3,1,3,2,1] -> [3,1,2].

C12. First non-repeating character [Easy]

Return the index of the first character that occurs exactly once, or -1. Example: 'leetcode' -> 0; 'aabb' -> -1.

C13. Valid anagram [Easy]

Return True if b is a rearrangement of a. Example: 'listen', 'silent' -> True.

C14. Top-k most common characters [Easy]

Return the k most frequent characters of s, most frequent first. Example: ('aabbcc', 2) -> ['b', 'a'].

C15. Group anagrams [Medium]

Group words that are anagrams of each other. Return a list of groups, each sorted, and the list of groups sorted. Example: ['eat', 'tea', 'tan', 'ate', 'nat', 'bat'] -> [['ate', 'eat', 'tea'], ['bat'], ['nat', 'tan']].

C16. Two Sum [Medium]

Return the indices [i, j] (i < j) of two numbers that add to target, or None. Example: ([2,7,11,15], 9) -> [0,1].

C17. Subarray sum equals k [Medium]

Count the contiguous subarrays whose sum equals k (values may be negative). Example: ([1,1,1], 2) -> 2.

C18. Majority element [Medium]

Return the element appearing more than $n/2$ times (it is guaranteed to exist). Example: `[2,2,1,1,1,2,2]` -> 2.

C19. Top-k frequent elements [Medium]

Return the k most frequent values; break ties by smaller value first. Example: `([1,1,1,2,2,3], 2)` -> `[1,2]`.

C20. Sort by frequency, then value [Medium]

Sort the list so that more frequent values come first; equal frequencies are ordered by value ascending. Example: `[4,5,6,5,4,3]` -> `[4,4,5,5,3,6]`.

C21. Sort by parity [Easy]

Put even numbers before odd numbers, keeping the relative order within each group. Example: `[3,1,2,4]` -> `[2,4,3,1]`.

C22. Rank students [Medium]

Given (name, mark) pairs, sort by mark descending, then name ascending. Example: `[('Bob', 85), ('Ann', 90), ('Cid', 90)]` -> `[('Ann', 90), ('Cid', 90), ('Bob', 85)]`.

C23. Largest number [Hard]

Arrange non-negative integers to form the largest possible number, returned as a string. Example: `[3,30,34,5,9]` -> `'9534330'`.

C24. Valid parentheses [Easy]

Given a string of `()[]{}` , return True if brackets are closed in the correct order. Example: `'({[]})'` -> True; `'()'` -> False.

C25. Next greater element [Medium]

For each element, return the next larger element to its right, or -1. Example: `[2,1,5,3,4]` -> `[5,5,-1,4,-1]`.

C26. Daily temperatures [Medium]

For each day, return how many days you must wait for a warmer temperature (0 if never). Example: `[73,74,75,71,69,72,76,73]` -> `[1,1,4,2,1,1,0,0]`.

C27. Min stack [Medium]

Design a stack with `push(x)`, `pop()`, `top()`, and `get_min()` all in $O(1)$. Example: push 5, push 3, push 7 -> `get_min()` is 3; pop -> `get_min()` still 3; pop, pop -> 5 at the bottom.

C28. Evaluate reverse Polish notation [Medium]

Evaluate an RPN expression given as tokens; division truncates toward zero. Example: `['2', '1', '+', '3', '*']` -> 9.

C29. Shortest path in a grid [Medium]

`grid[r][c]` is 0 (open) or 1 (wall). Return the number of steps from the top-left to the bottom-right moving up/down/left/right, or -1. Example: `[[0,0,0], [1,1,0], [0,0,0]]` -> 4.

C30. Binary tree level order [Medium]

Return the values of a binary tree level by level as a list of lists. Example: tree 3 -> (9, 20 -> (15, 7)) -> `[[3], [9,20], [15,7]]`.

C31. Sliding window maximum [Hard]

Return the maximum of every window of size k. Example: `([1,3,-1,-3,5,3,6,7], 3)` -> `[3,3,5,5,6,7]`.

C32. Moving average [Medium]

Implement `MovingAverage(k)` with `next(x)` returning the average of the last k values seen. Example: k=3: 1 -> 1.0, 10 -> 5.5, 3 -> 4.666..., 5 -> 6.0.

C33. K-th largest element [Medium]

Return the k-th largest value (k = 1 is the maximum). Example: `([3,2,1,5,6,4], 2)` -> 5.

C34. Merge k sorted lists [Hard]

Merge k sorted lists into one sorted list. Example: `[[1,4,5], [1,3,4], [2,6]]` -> `[1,1,2,3,4,4,5,6]`.

C35. K closest points to the origin [Medium]

Return the k points closest to (0, 0) (Euclidean), closest first. Example: (`[[3,3],[5,-1],[-2,4]]`, 2) -> `[[3,3],[-2,4]]`.

C36. Search insert position [Easy]

In a sorted list, return the index where x is found or would be inserted. Example: (`[1,3,5,6]`, 2) -> 1.

C37. Count occurrences in a sorted list [Medium]

Count how many times x appears in a sorted list in $O(\log n)$. Example: (`[1,2,2,2,3]`, 2) -> 3.

C38. Longest increasing subsequence (length) [Hard]

Return the length of the longest strictly increasing subsequence. Example: `[10,9,2,5,3,7,101,18]` -> 4.

C39. Minimum meeting rooms [Medium]

Given `[start, end)` intervals, return the minimum number of rooms needed. Example: `[[0,30],[5,10],[15,20]]` -> 2.

C40. Running median [Hard]

Implement MedianFinder with `add(x)` and `median()` for a stream of numbers. Example: add 1, add 2 -> median 1.5; add 3 -> median 2.0.

8. Complete Solutions

8.1 Conceptual Answers

- Q1.** What is a list comprehension, and why is it preferred over a for + append loop?
A single expression `[expr for x in it if cond]` that builds a list. It is shorter, states intent ("the list *is* these values"), avoids repeated append lookups so it is usually faster, and keeps the loop variable local. Use a normal loop when you need break, try/except, or side effects.
- Q2.** In a comprehension, what is the difference between `if` at the end and `x if c else y` at the front?
`if` at the end is a **filter** (items failing the test are dropped, result may be shorter). `x if c else y` at the front is a **transform** (every item is kept, its value depends on the condition; `else` is mandatory). `[x for x in a if c else y]` is a `SyntaxError`.
- Q3.** Why is `[[0] * 3] * 3` a bug, and how do you fix it?
`*` repeats the **reference**, so all three rows are the same inner list; changing one cell changes every row. Fix: `[[0] * 3 for _ in range(3)]`, which evaluates `[0] * 3` afresh for each row.
- Q4.** Does the loop variable of a comprehension leak into the surrounding scope in Python 3?
No. The comprehension has its own scope, so after `[i for i in range(3)]` the name `i` is unchanged (or undefined). (Python 2 list comprehensions leaked; generator expressions never did.)
- Q5.** When should you use a generator expression instead of a list comprehension?
When you only need to iterate once and do not need the list itself - e.g. `sum(...)`, `any(...)`, `max(...)`, `''.join(...)`. It uses $O(1)$ memory and `any/all` can stop early. Use a list when you need indexing, `len`, or several passes.
- Q6.** What does `{}` create? How do you create an empty set?
`{}` is an empty **dict**. An empty set is `set()`. `{1, 2}` is a set and `{1: 2}` is a dict, so only the empty literal is ambiguous.
- Q7.** What happens with duplicate keys in a dict comprehension?
Later pairs overwrite earlier ones, so only the last value per key survives. Inverting a dict whose values repeat therefore loses entries; use `defaultdict(list)` to group instead.
- Q8.** Why can you not iterate twice over the result of `map`, `filter`, `zip`, or `enumerate`?
They return **iterators**, which are exhausted after one pass. The second loop (or `list()`) sees nothing. Convert to a list first if you need several passes. `range` is different: it is a re-iterable sequence.
- Q9.** Difference between `sorted()` and `list.sort()`?
`sorted(iterable)` returns a **new list** and accepts any iterable; `lst.sort()` sorts a list **in place** and returns `None`. Consequently `a = a.sort()` sets `a` to `None`.
- Q10.** What does it mean that Python's sort is stable, and how do you use it?
Items with equal keys keep their original relative order. You can sort by a secondary key first and a primary key second (two passes) to get multi-level ordering, or use a tuple key for one pass.
- Q11.** How many times is the key function called per element, and what does `max(items, key=f)` return?
key is called **once per element**. `max` returns the **item** whose key is largest (the first one on ties), not the key value.
- Q12.** Why is `round(2.5)` equal to 2?
Python 3 uses round-half-to-even (banker's rounding) for exact ties: `round(2.5) == 2`, `round(3.5) == 4`. Also, some decimals (like 2.675) are stored slightly off in binary floating point, so `round(2.675, 2)` gives 2.67.
- Q13.** What do `any([])` and `all([])` return, and why?
`any([])` is `False` (no element is truthy) and `all([])` is `True` (vacuous truth - no element violates the condition).
- Q14.** What happens with `max([])` and how do you avoid it?
`ValueError: max() arg is an empty sequence`. Pass `default=: max(a, default=0)` (or check if `a:`). `sum([])` is fine and returns 0.

- Q15.** Why does `isinstance(True, int)` return `True`?
`bool` is a subclass of `int` (`True == 1, False == 0`). To accept integers but reject booleans, test `isinstance(x, int)` and `not isinstance(x, bool)`.
- Q16.** Explain `is` versus `==`. When is `is` correct?
`==` compares values (`__eq__`); `is` compares object identity (same object in memory, `id(a) == id(b)`). Use `is` for `None` (`x is None`) and for the singletons `True/False`. Never use `is` to compare numbers or strings.
- Q17.** How do `ord()` and `chr()` help in string problems?
`ord(c)` gives the code point and `chr(n)` the character. Letter arithmetic: `ord(c) - ord('a')` maps a-z to 0-25 (indexing a 26-slot array) and `chr((ord(c) - ord('a') + k) % 26 + ord('a'))` shifts letters (Caesar cipher).
- Q18.** What does `zip` do with iterables of different lengths? How can you change that?
It stops at the shortest silently. Use `zip(..., strict=True)` (Python 3.10+) to raise `ValueError`, or `itertools.zip_longest` to pad the shorter ones.
- Q19.** Why prefer `enumerate` to `range(len(a))`?
It yields `(index, value)` directly, removing manual indexing and a class of off-by-one bugs, and it works on any iterable (not just sequences). `start=` sets the first index.
- Q20.** What is the difference between `*args` and `**kwargs`?
`*args` collects extra **positional** arguments into a **tuple**; `**kwargs` collects extra **keyword** arguments into a **dict**. At a call site `*` unpacks an iterable into positional arguments and `**` unpacks a dict into keyword arguments.
- Q21.** What does the starred name do in `first, *rest = items`?
It collects all remaining items into a **list** (possibly empty). Only one starred target is allowed per assignment; it may appear anywhere: `first, *mid, last = items`.
- Q22.** Why does `a, b = b, a` swap correctly without a temp variable?
The right-hand side `(b, a)` is fully evaluated into a tuple **first**; then its items are assigned to the left targets. So neither old value is lost.
- Q23.** What is the syntax of the ternary expression, and when is `x` or `default` a bad substitute?
`value_if_true if condition else value_if_false`. `x` or `default` picks the default for **any falsy** `x` (`0, '', []`), not just `None`; if `0` is a legitimate value, use `x if x is not None else default`.
- Q24.** What are the limits of `lambda`, and what is the late-binding trap?
A `lambda` body is a single expression (no statements or assignments). Free variables are looked up **when the lambda is called**, so `[lambda: i for i in range(3)]` all return the final `i`. Bind early with a default argument: `lambda i=i: i`.
- Q25.** What is `Counter`, and how does it differ from a normal dict?
A dict subclass that counts hashable items. Missing keys return `0` (no `KeyError`), it has `most_common(k)`, `update` **adds** counts, and supports `+` `-` `&` `|` between counters. Build cost `O(n)`.
- Q26.** What is the danger of reading a missing key of a `defaultdict`?
Merely reading `d[k]` **creates** `k` with the default value, growing the dict and making `k in d` become `True`. Use `k in d` or `d.get(k)` to test without inserting.
- Q27.** When should you choose `defaultdict` over `Counter`, and vice versa?
`Counter` when you just count an iterable (one line, `most_common`). `defaultdict` when grouping or accumulating other kinds of values - lists (`defaultdict(list)`), sets, sums, nested dicts, adjacency lists.
- Q28.** Why is `deque` better than a list for a queue?
`list.pop(0)` and `list.insert(0, x)` shift all elements, costing `O(n)`. `deque.popleft()` and `appendleft()` are `O(1)`. BFS on large graphs with a list-based queue can time out.
- Q29.** Is `heapq` a min-heap or max-heap? How do you get max-heap behaviour?
A **min-heap**: `heap[0]` is the smallest. For max-heap behaviour push negated numbers (`-x`) and negate on pop, or store `(-priority, item)` tuples. There is no public max-heap API.

Q30. Is a heap list sorted? What do you know about `h[1]` and `h[2]`?

No - it only satisfies the heap invariant ($h[i] \leq h[2i+1]$ and $h[i] \leq h[2i+2]$). Only `h[0]` is guaranteed to be the minimum. Pop repeatedly to get sorted order.

Q31. What goes wrong with `heappush(h, (priority, obj))` and how do you fix it?

When two priorities tie, Python compares `obj` next; if objects are not orderable it raises `TypeError`. Fix with a tie-breaker (`priority, next(counter), obj`) or by making the objects comparable.

Q32. Compare `heapify(lst)` with pushing `n` items one at a time.

`heapify` builds a heap in place in **$O(n)$** ; `n` individual pushes cost **$O(n \log n)$** . Use `heapify` when you already have all the data.

Q33. When is `heapq.nlargest(k, a)` better than `sorted(a, reverse=True)[:k]`?

When `k` is much smaller than `n`: it runs in $O(n \log k)$ with $O(k)$ memory versus $O(n \log n)$ time and $O(n)$ memory. For `k` close to `n`, sorting is simpler and as fast.

Q34. What is the difference between `bisect_left` and `bisect_right`? What must be true of the list?

For equal items, `bisect_left` returns the index **before** them (first item $\geq x$); `bisect_right` the index **after** them (first item $> x$). The list must already be **sorted**, otherwise results are meaningless.

Q35. Is `bisect.insort` $O(\log n)$? Explain.

Finding the position is $O(\log n)$, but inserting into a Python list shifts elements, so `insort` is **$O(n)$** overall. For heavy insert workloads use a heap or a balanced structure (e.g. `sortedcontainers`).

Q36. How do you remove duplicates while preserving order, and why is `list(set(a))` not enough?

`list(dict.fromkeys(a))` (dicts keep insertion order, first occurrence wins). `set` is unordered so `list(set(a))` may reorder items. Items must be hashable.

Q37. Why are dict and set lookups $O(1)$ on average, and what does "hashable" mean?

They use a hash table: `hash(key)` gives a slot. A hashable object has a hash that never changes and can be compared with `==` - `int`, `str`, tuple of hashables. Lists, dicts, and sets are mutable and therefore unhashable. Worst case $O(n)$ with many collisions.

Q38. Explain why Two Sum stores the value in the dict **after** checking for the complement.

Checking first guarantees the current element is not paired with itself. Storing value $\rightarrow$ index after the check also handles duplicates like `[3, 3]`, target 6, because the first 3 is stored before the second is examined. Time $O(n)$, space $O(n)$.

Q39. Give three problems solved by a stack and state why a list is enough as a stack in Python.

Valid parentheses, next greater element (monotonic stack), evaluating reverse Polish notation (also undo, iterative DFS). `list.append` and `list.pop()` are $O(1)$ amortised at the end, so no special class is needed.

Q40. In BFS with a queue, why mark nodes visited when they are enqueued instead of when dequeued?

If you mark on dequeue, the same node can be enqueued many times by different neighbours, wasting time and memory (up to $O(E)$ duplicates). Marking on enqueue guarantees each node enters the queue once: $O(V + E)$.

8.2 Output Answers (every output was generated by running the code)

01.

```
print([x for x in range(10) if x % 3 == 0])
```

Output:

```
[0, 3, 6, 9]
```

Why: The `if` at the end is a filter: it keeps 0, 3, 6, 9.

O2.

```
print([x if x % 2 else -x for x in range(5)])
```

Output:

```
[0, 1, -2, 3, -4]
```

Why: Ternary at the front transforms every item. $x \% 2$ is truthy for odd x , so odd stay, even become negative (-0 is 0).

O3.

```
g = [[0] * 2] * 2
g[0][0] = 1
print(g)
```

Output:

```
[[1, 0], [1, 0]]
```

Why: Both rows are the same list object, so the change shows in both rows.

O4.

```
g = [[0] * 2 for _ in range(2)]
g[0][0] = 1
print(g)
```

Output:

```
[[1, 0], [0, 0]]
```

Why: The comprehension makes a fresh inner list per row, so only row 0 changes.

O5.

```
print({x % 3: x for x in range(7)})
```

Output:

```
{0: 6, 1: 4, 2: 5}
```

Why: Duplicate keys overwrite: the last x per key wins ($0 \rightarrow 6$, $1 \rightarrow 4$, $2 \rightarrow 5$).

O6.

```
print(type({}), type({1}), type({1: 2}))
```

Output:

```
<class 'dict'> <class 'set'> <class 'dict'>
```

Why: $\{\}$ is a dict, $\{1\}$ is a set, $\{1: 2\}$ is a dict.

O7.

```
i = 100
result = [i for i in range(3)]
print(i)
```

Output:

```
100
```

Why: The comprehension variable is local to the comprehension in Python 3, so the outer i stays 100.

O8.

```
print(sum(x for x in range(5) if x % 2))
```

Output:

```
4
```

Why: Generator of odd numbers 1 and 3; the sum is 4.

09.

```
m = map(int, ["1", "2"])
print(list(m))
print(list(m))
```

Output:

```
[1, 2]
[]
```

Why: map returns a one-shot iterator: the first list consumes it, the second is empty.

010.

```
print(list(zip([1, 2, 3], "ab")))
print(list(zip(*[[1, 2], [3, 4]])))
```

Output:

```
[(1, 'a'), (2, 'b')]
[(1, 3), (2, 4)]
```

Why: zip stops at the shortest input. zip(*matrix) transposes and yields tuples.

011.

```
print(round(2.5), round(3.5), round(-0.5), round(1.25, 1))
```

Output:

```
2 4 0 1.2
```

Why: Ties round to the even neighbour: 2, 4, 0 (i.e. -0 as int), and 1.25 -> 1.2.

012.

```
print(any([]), all([]), any([0, ""]), all([1, "a"]))
```

Output:

```
False True False True
```

Why: any([]) False; all([]) True; no truthy item in [0, '']; both items truthy.

013.

```
print(max("apple", "banana"), max(3, 10, 7), min([], default="none"))
```

Output:

```
banana 10 none
```

Why: Strings compare lexicographically ('b' > 'a'); the 3-argument form returns 10; default avoids the error on an empty input.

014.

```
print(max([3, 1, 2], key=lambda x: -x))
```

Output:

```
1
```

Why: The key is -x; the largest key is -1 (from x = 1), and max returns the **item**: 1.

015.

```
a = [3, 1, 2]
b = a.sort()
print(a, b)
```

Output:

```
[1, 2, 3] None
```

Why: .sort() sorts in place and returns None.

O16.

```
print(sorted("banana"))
```

Output:

```
['a', 'a', 'a', 'b', 'n', 'n']
```

Why: sorted of a string returns a **list** of characters.

O17.

```
print(sorted([3, -1, 2, -5], key=abs))
```

Output:

```
[-1, 2, 3, -5]
```

Why: Keys are 3, 1, 2, 5 -> order by magnitude: -1, 2, 3, -5 (original values are kept).

O18.

```
words = ["bb", "a", "cc", "d"]  
print(sorted(words, key=len))
```

Output:

```
['a', 'd', 'bb', 'cc']
```

Why: Stable sort: equal keys keep original order, so 'a' before 'd' and 'bb' before 'cc'.

O19.

```
print(list(enumerate("ab", start=5)))
```

Output:

```
[(5, 'a'), (6, 'b')]
```

Why: enumerate with start=5 yields (5, 'a') and (6, 'b').

O20.

```
print(list(range(10, 0, -3)), len(range(2, 20, 5)), 9 in range(0, 20, 3))
```

Output:

```
[10, 7, 4, 1] 4 True
```

Why: 10, 7, 4, 1 (0 excluded); 2, 7, 12, 17 has 4 items; 9 is in 0, 3, 6, 9, ...

O21.

```
print(ord("a") + 1, chr(ord("a") + 1), ord("Z") - ord("A"))
```

Output:

```
98 b 25
```

Why: ord('a') is 97, so 98; chr(98) is 'b'; 'Z' is 25 letters after 'A'.

O22.

```
a, *b, c = range(5)  
print(a, b, c)
```

Output:

```
0 [1, 2, 3] 4
```

Why: a takes the first, c the last, the star collects the middle **list**.

O23.

```
a, *b = "hi"  
print(a, b)
```

Output:

```
h ['i']
```

Why: Any iterable unpacks: 'h' goes to a, the star gives the **list** ['i'].

O24.

```
def f(*args, **kw):  
    return args, kw  
  
print(f(1, 2, x=3))
```

Output:

```
((1, 2), {'x': 3})
```

Why: *args is a tuple of positionals, **kw a dict of keyword arguments.

O25.

```
print(["*ab", *[1, 2]], {**{"a": 1}, **{"a": 2, "b": 3}})
```

Output:

```
['a', 'b', 1, 2] {'a': 2, 'b': 3}
```

Why: * merges iterables into a list; ** merges dicts and later keys win.

O26.

```
x = [1, 2]  
y = x  
y += [3]  
y = y + [4]  
print(x, y)
```

Output:

```
[1, 2, 3] [1, 2, 3, 4]
```

Why: += extends the **same** list in place (so x sees 3); y + [4] creates a new list bound only to y.

O27.

```
print(5 if 0 else 6, 0 or "d", "" or [] or "z", 3 and 4)
```

Output:

```
6 d z 4
```

Why: Ternary picks 6. or returns the first truthy operand (or the last), and returns the first falsy operand or the last (4).

O28.

```
fs = [lambda: i for i in range(3)]  
print([f() for f in fs])
```

Output:

```
[2, 2, 2]
```

Why: Late binding: each lambda reads i when called, after the loop ended with i = 2.

O29.

```
fs = [lambda i=i: i for i in range(3)]
print([f() for f in fs])
```

Output:

```
[0, 1, 2]
```

Why: The default argument `i=i` captures the current value at definition time.

O30.

```
from collections import Counter
c = Counter("aabbbc")
print(c.most_common(2), c["z"], len(c))
```

Output:

```
[('b', 3), ('a', 2)] 0 3
```

Why: Top two are b (3) and a (2). A missing key gives 0 without being added, so there are still 3 keys.

O31.

```
from collections import Counter
c = Counter("abc")
c.subtract("aab")
print(c)
```

Output:

```
Counter({'c': 1, 'b': 0, 'a': -1})
```

Why: subtract allows zero and negative counts: `a = 1-2 = -1`, `b = 1-1 = 0`, `c = 1`. Counter prints highest count first.

O32.

```
from collections import defaultdict
d = defaultdict(list)
d["x"]
print(len(d), "y" in d, d.get("y"))
```

Output:

```
1 False None
```

Why: Reading `d['x']` created it (length 1). `'y' in d` and `.get('y')` do not create keys.

O33.

```
from collections import deque
q = deque([1, 2, 3])
q.rotate(1)
q.appendleft(0)
print(q, q.pop())
```

Output:

```
deque([0, 3, 1]) 2
```

Why: Arguments are evaluated left to right, but `q` is the same object: `q.pop()` runs **before** printing, so the printed deque is already missing the popped 2.

O34.

```
from collections import deque
q = deque(maxlen=2)
for i in range(5):
    q.append(i)
print(q)
```

Output:

```
deque([3, 4], maxlen=2)
```

Why: With maxlen=2 each append past capacity drops the oldest item, leaving the last two values.

O35.

```
import heapq
h = [5, 1, 4]
heapq.heapify(h)
print(h[0], heapq.heappop(h), h[0])
```

Output:

```
1 1 4
```

Why: After heapify the smallest (1) is at index 0. The arguments run in order: peek 1, pop 1, then the new minimum 4.

O36.

```
import heapq
h = []
for x in [3, 1, 2]:
    heapq.heappush(h, -x)
print(-h[0])
```

Output:

```
3
```

Why: Pushing negatives turns the min-heap into a max-heap: the smallest negative is -3, so the largest original is 3.

O37.

```
import heapq
print(heapq.nlargest(2, [4, 1, 9, 9, 3]), heapq.nsmallest(2, "hello"))
```

Output:

```
[9, 9] ['e', 'h']
```

Why: Top two largest keep duplicates (9, 9). nsmallest works on strings: 'e' then 'h'.

O38.

```
import bisect
a = [1, 3, 3, 5]
print(bisect.bisect_left(a, 3), bisect.bisect_right(a, 3), bisect.bisect_left(a, 4), bisect.bisect_left(a, 0))
```

Output:

```
1 3 3 0
```

Why: left of the 3s is index 1, right of them is 3, 4 belongs before 5 (index 3), 0 belongs at the start.

O39.

```
a = [1, 2]
b = [1, 2]
print(a == b, a is b, a is a[:], isinstance(True, int))
```

Output:

```
True False False True
```

Why: Equal values but different objects; a slice copy is a new object; bool is a subclass of int.

O40.

```
print(-7 // 2, -7 % 2, 7 // -2, divmod(-7, 2))
```

Output:

```
-4 1 -4 (-4, 1)
```

Why: Floor division rounds toward negative infinity: $-7 // 2 = -4$, remainder 1 (sign of divisor). (C++ would give -3 and -1.)

8.3 Coding Solutions (every solution was tested)

C1. Squares of evens [Easy]

Return a list of the squares of the even numbers in a, keeping order. Example: [1, 2, 3, 4] -> [4, 16].

```
def squares_of_evens(a):
    return [x * x for x in a if x % 2 == 0]
```

Explanation and complexity: Filter at the end, transform at the front. $O(n)$ time, $O(n)$ space.

C2. Flatten a matrix [Easy]

Flatten a list of lists into one list, row by row. Example: [[1, 2], [3], []] -> [1, 2, 3].

```
def flatten(m):
    return [x for row in m for x in row]
```

Explanation and complexity: Two for clauses in the same order as nested loops (outer first). $O(\text{total items})$.

C3. Transpose [Easy]

Transpose a rectangular matrix. Example: [[1,2,3],[4,5,6]] -> [[1,4],[2,5],[3,6]].

```
def transpose(m):
    return [list(col) for col in zip(*m)]
```

Explanation and complexity: `zip(*m)` pairs the i -th element of every row. Wrap each tuple in list. $O(r*c)$.

C4. Invert a dictionary with repeated values [Easy]

Given a dict, return a dict mapping each value to the **list of keys** that had it (in original order).

Example: {'a':1, 'b':2, 'c':1} -> {1:['a', 'c'], 2:['b']}.

```
from collections import defaultdict

def invert_grouped(d):
    out = defaultdict(list)
    for k, v in d.items():
        out[v].append(k)
    return dict(out)
```

Explanation and complexity: A dict comprehension would lose keys with repeated values. `defaultdict(list)` groups them. $O(n)$.

C5. Count vowels [Easy]

Count the vowels (aeiou, case-insensitive) in a string. Example: 'Education' -> 5.

```
def count_vowels(s):
    return sum(ch in "aeiou" for ch in s.lower())
```

Explanation and complexity: Booleans add as 0/1 so sum counts matches. O(n).

C6. Reverse the words [Easy]

Reverse the order of words in a sentence, collapsing extra spaces. Example: ' hello world ' -> 'world hello'.

```
def reverse_words(s):
    return " ".join(reversed(s.split()))
```

Explanation and complexity: split() with no argument splits on any run of whitespace. join rebuilds. O(n).

C7. Palindrome (alphanumeric only) [Easy]

Return True if the string reads the same forwards and backwards, ignoring case and non-alphanumeric characters. Example: 'A man, a plan, a canal: Panama' -> True.

```
def is_palindrome(s):
    t = [c.lower() for c in s if c.isalnum()]
    return t == t[::-1]
```

Explanation and complexity: Filter then compare with the reverse. O(n) time and space (a two-pointer version uses O(1) space).

C8. Second largest distinct value [Easy]

Return the second largest **distinct** value, or None if it does not exist. Example: [5, 5, 3] -> 3; [7] -> None. Do it in one pass.

```
def second_largest(a):
    first = second = None
    for x in a:
        if first is None or x > first:
            first, second = x, first
        elif x != first and (second is None or x > second):
            second = x
    return second
```

Explanation and complexity: Tuple assignment updates both trackers at once. O(n) time, O(1) space, and no wrong initial value (None sentinel).

C9. Move zeros to the end [Easy]

Return a list with all zeros moved to the end, other elements keeping their order. Example: [0,1,0,3,12] -> [1,3,12,0,0].

```
def move_zeros(a):
    return sorted(a, key=lambda x: x == 0)
```

Explanation and complexity: Key is False (0) for non-zeros and True (1) for zeros; a **stable** sort keeps the relative order. O(n log n) (a two-pointer version is O(n)).

C10. Is the list sorted? [Easy]

Return True if the list is in non-decreasing order. Example: [1,2,2,5] -> True.

```
def is_sorted(a):
    return all(x <= y for x, y in zip(a, a[1:]))
```

Explanation and complexity: zip(a, a[1:]) yields adjacent pairs; all short-circuits on the first violation. O(n) (slicing copies, use itertools.pairwise on 3.10+ to avoid that).

C11. Remove duplicates, keep order [Easy]

Remove duplicates from a list of hashable items keeping the first occurrence. Example: [3,1,3,2,1] -> [3,1,2].

```
def dedupe(a):
    return list(dict.fromkeys(a))
```

Explanation and complexity: Dict keys are unique and ordered by insertion. O(n) time and space.

C12. First non-repeating character [Easy]

Return the index of the first character that occurs exactly once, or -1. Example: 'leetcode' -> 0; 'aabb' -> -1.

```
from collections import Counter

def first_unique(s):
    c = Counter(s)
    for i, ch in enumerate(s):
        if c[ch] == 1:
            return i
    return -1
```

Explanation and complexity: Count once, then scan in original order. O(n) time, O(k) space.

C13. Valid anagram [Easy]

Return True if b is a rearrangement of a. Example: 'listen', 'silent' -> True.

```
from collections import Counter

def is_anagram(a, b):
    return Counter(a) == Counter(b)
```

Explanation and complexity: Counters compare as dicts of counts. O(n). Alternative: sorted(a) == sorted(b) in O(n log n).

C14. Top-k most common characters [Easy]

Return the k most frequent characters of s, most frequent first. Example: ('aabbbc', 2) -> ['b', 'a'].

```
from collections import Counter

def top_k_chars(s, k):
    return [ch for ch, _ in Counter(s).most_common(k)]
```

Explanation and complexity: most_common(k) returns (item, count) pairs sorted by count. O(n + m log k) for m distinct characters.

C15. Group anagrams [Medium]

Group words that are anagrams of each other. Return a list of groups, each sorted, and the list of groups sorted. Example: ['eat', 'tea', 'tan', 'ate', 'nat', 'bat'] -> [['ate', 'eat', 'tea'], ['bat'], ['nat', 'tan']].

```
from collections import defaultdict

def group_anagrams(words):
    groups = defaultdict(list)
    for w in words:
        groups["".join(sorted(w))].append(w)
    return sorted(sorted(g) for g in groups.values())
```

Explanation and complexity: The sorted letters are a canonical key shared by all anagrams. O(n * k log k) for n words of length k.

C16. Two Sum [Medium]

Return the indices $[i, j]$ ($i < j$) of two numbers that add to target, or None. Example: $([2, 7, 11, 15], 9)$ -> $[0, 1]$.

```
def two_sum(nums, target):
    seen = {} # value -> index
    for i, x in enumerate(nums):
        need = target - x
        if need in seen:
            return [seen[need], i]
        seen[x] = i
    return None
```

Explanation and complexity: One pass with a hash map. Store **after** checking so an element never pairs with itself. $O(n)$ time, $O(n)$ space.

C17. Subarray sum equals k [Medium]

Count the contiguous subarrays whose sum equals k (values may be negative). Example: $([1, 1, 1], 2)$ -> 2.

```
from collections import Counter

def subarray_sum_k(nums, k):
    seen = Counter({0: 1}) # prefix sum -> how many times seen
    total = count = 0
    for x in nums:
        total += x
        count += seen[total - k] # Two-Sum idea on prefix sums
        seen[total] += 1
    return count
```

Explanation and complexity: A subarray sums to k when $\text{prefix}[j] - \text{prefix}[i] == k$. Count earlier prefixes equal to $\text{prefix}[j] - k$. $O(n)$ time, $O(n)$ space.

C18. Majority element [Medium]

Return the element appearing more than $n/2$ times (it is guaranteed to exist). Example: $[2, 2, 1, 1, 1, 2, 2]$ -> 2.

```
from collections import Counter

def majority(a):
    return Counter(a).most_common(1)[0][0]
```

Explanation and complexity: $O(n)$ time with $O(k)$ space. (Boyer-Moore voting solves it in $O(1)$ space.)

C19. Top-k frequent elements [Medium]

Return the k most frequent values; break ties by smaller value first. Example: $([1, 1, 1, 2, 2, 3], 2)$ -> $[1, 2]$.

```
import heapq
from collections import Counter

def top_k_frequent(nums, k):
    freq = Counter(nums)
    best = heapq.nsmallest(k, freq.items(), key=lambda kv: (-kv[1], kv[0]))
    return [x for x, _ in best]
```

Explanation and complexity: A tuple key $(-\text{count}, \text{value})$ encodes both rules. `nsmallest` is $O(n \log k)$ - better than a full sort when k is small.

C20. Sort by frequency, then value [Medium]

Sort the list so that more frequent values come first; equal frequencies are ordered by value ascending.
Example: [4,5,6,5,4,3] -> [4,4,5,5,3,6].

```
from collections import Counter

def sort_by_frequency(a):
    freq = Counter(a)
    return sorted(a, key=lambda x: (-freq[x], x))
```

Explanation and complexity: Negating the count reverses only that criterion. $O(n \log n)$.

C21. Sort by parity [Easy]

Put even numbers before odd numbers, keeping the relative order within each group. Example:
[3,1,2,4] -> [2,4,3,1].

```
def sort_by_parity(a):
    return sorted(a, key=lambda x: x % 2)
```

Explanation and complexity: Key 0 for evens, 1 for odds; stable sort preserves order inside each group.
 $x \% 2$ is never -1 in Python, even for negatives.

C22. Rank students [Medium]

Given (name, mark) pairs, sort by mark descending, then name ascending. Example:
[('Bob',85), ('Ann',90), ('Cid',90)] -> [('Ann',90), ('Cid',90), ('Bob',85)].

```
def rank_students(students):
    return sorted(students, key=lambda s: (-s[1], s[0]))
```

Explanation and complexity: Tuple key with a negated number. $O(n \log n)$.

C23. Largest number [Hard]

Arrange non-negative integers to form the largest possible number, returned as a string. Example:
[3,30,34,5,9] -> '9534330'.

```
from functools import cmp_to_key

def largest_number(nums):
    def cmp(a, b):
        if a + b > b + a:
            return -1
        if a + b < b + a:
            return 1
        return 0
    strs = sorted(map(str, nums), key=cmp_to_key(cmp))
    res = "".join(strs)
    return "0" if res[0] == "0" else res
```

Explanation and complexity: No single-item key expresses this order, so use a comparator that puts a first when $a+b > b+a$ (string concatenation). $O(n \log n * L)$.

C24. Valid parentheses [Easy]

Given a string of ()[]{}, return True if brackets are closed in the correct order. Example: '({[]})' -> True; '()' -> False.

```
def is_valid(s):
    pairs = {"(": ")", "[": "]", "{": "}"
    stack = []
    for ch in s:
        if ch in pairs:
            if not stack or stack[-1] != pairs[ch]:
                return False
            stack.pop()
        else:
            stack.append(ch)
    return not stack
```

Explanation and complexity: Push openers; on a closer the top of the stack must be its partner. End with an empty stack. $O(n)$.

C25. Next greater element [Medium]

For each element, return the next larger element to its right, or -1. Example: [2,1,5,3,4] -> [5,5,-1,4,-1].

```
def next_greater(a):
    res = [-1] * len(a)
    st = [] # indices still waiting for an answer
    for i, x in enumerate(a):
        while st and a[st[-1]] < x:
            res[st.pop()] = x
        st.append(i)
    return res
```

Explanation and complexity: Monotonic (decreasing) stack: each index is pushed and popped once, so $O(n)$.

C26. Daily temperatures [Medium]

For each day, return how many days you must wait for a warmer temperature (0 if never). Example: [73,74,75,71,69,72,76,73] -> [1,1,4,2,1,1,0,0].

```
def daily_temperatures(t):
    res = [0] * len(t)
    st = []
    for i, x in enumerate(t):
        while st and t[st[-1]] < x:
            j = st.pop()
            res[j] = i - j
        st.append(i)
    return res
```

Explanation and complexity: Same monotonic-stack pattern as the previous problem, storing the **distance** instead of the value. $O(n)$.

C27. Min stack [Medium]

Design a stack with push(x), pop(), top(), and get_min() all in $O(1)$. Example: push 5, push 3, push 7 -> get_min() is 3; pop -> get_min() still 3; pop, pop -> 5 at the bottom.

```
class MinStack:
    def __init__(self):
        self.st = [] # pairs (value, minimum_so_far)

    def push(self, x):
        m = min(x, self.st[-1][1]) if self.st else x
        self.st.append((x, m))

    def pop(self):
        return self.st.pop()[0]

    def top(self):
        return self.st[-1][0]

    def get_min(self):
        return self.st[-1][1]
```

Explanation and complexity: Store the running minimum next to every value so popping restores the previous minimum automatically. $O(1)$ per operation, $O(n)$ space.

C28. Evaluate reverse Polish notation [Medium]

Evaluate an RPN expression given as tokens; division truncates toward zero. Example:

['2', '1', '+', '3', '*'] -> 9.

```
def eval_rpn(tokens):
    st = []
    for t in tokens:
        if t in {"+", "-", "*", "/"}:
            b = st.pop()
            a = st.pop()
            if t == "+": st.append(a + b)
            elif t == "-": st.append(a - b)
            elif t == "*": st.append(a * b)
            else: st.append(int(a / b))      # truncate toward zero
        else:
            st.append(int(t))
    return st[0]
```

Explanation and complexity: Operands go on the stack; an operator pops two (note the order: second popped is the left operand). `int(a / b)` truncates toward zero, unlike `//` which floors. $O(n)$.

C29. Shortest path in a grid [Medium]

`grid[r][c]` is 0 (open) or 1 (wall). Return the number of steps from the top-left to the bottom-right moving up/down/left/right, or -1. Example: `[[0,0,0],[1,1,0],[0,0,0]]` -> 4.

```
from collections import deque

def shortest_path(grid):
    n, m = len(grid), len(grid[0])
    if grid[0][0] or grid[n - 1][m - 1]:
        return -1
    q = deque([(0, 0, 0)])
    seen = {(0, 0)}
    while q:
        r, c, d = q.popleft()
        if (r, c) == (n - 1, m - 1):
            return d
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < n and 0 <= nc < m and grid[nr][nc] == 0 and (nr, nc) not in seen:
                seen.add((nr, nc))
                q.append((nr, nc, d + 1))
    return -1
```

Explanation and complexity: BFS explores level by level, so the first arrival at the target is the shortest. Mark visited when enqueueing. $O(\text{rows} * \text{cols})$.

C30. Binary tree level order [Medium]

Return the values of a binary tree level by level as a list of lists. Example: tree 3 -> (9, 20 -> (15, 7)) -> [[3],[9,20],[15,7]].

```
from collections import deque

class TreeNode:
    def __init__(self, val, left=None, right=None):
        self.val, self.left, self.right = val, left, right

def level_order(root):
    if not root:
        return []
    out, q = [], deque([root])
    while q:
        level = []
        for _ in range(len(q)):          # size of THIS level, frozen now
            node = q.popleft()
            level.append(node.val)
            if node.left:
                q.append(node.left)
            if node.right:
                q.append(node.right)
        out.append(level)
    return out
```

Explanation and complexity: Capture `len(q)` at the start of each level. $O(n)$ time, $O(\text{width})$ space.

C31. Sliding window maximum [Hard]

Return the maximum of every window of size `k`. Example: `([1,3,-1,-3,5,3,6,7], 3) -> [3,3,5,5,6,7]`.

```
from collections import deque

def max_sliding_window(nums, k):
    dq, out = deque(), []
    for i, x in enumerate(nums):
        while dq and nums[dq[-1]] <= x:
            dq.pop()
        dq.append(i)
        if dq[0] <= i - k:
            dq.popleft()
        if i >= k - 1:
            out.append(nums[dq[0]])
    return out
```

Explanation and complexity: A monotonic deque keeps candidates in decreasing order, so the front is always the window maximum. Every index enters and leaves once: $O(n)$.

C32. Moving average [Medium]

Implement `MovingAverage(k)` with `next(x)` returning the average of the last `k` values seen. Example: `k=3: 1 -> 1.0, 10 -> 5.5, 3 -> 4.666..., 5 -> 6.0`.

```
from collections import deque

class MovingAverage:
    def __init__(self, k):
        self.w = deque(maxlen=k)
        self.total = 0

    def next(self, x):
        if len(self.w) == self.w.maxlen:
            self.total -= self.w[0]
        self.w.append(x)
        self.total += x
        return self.total / len(self.w)
```

Explanation and complexity: `deque(maxlen=k)` discards the oldest value automatically; keep a running total for $O(1)$ per call.

C33. K-th largest element [Medium]

Return the k-th largest value (k = 1 is the maximum). Example: ([3,2,1,5,6,4], 2) -> 5.

```
import heapq

def kth_largest(nums, k):
    h = []
    for x in nums:
        heapq.heappush(h, x)
        if len(h) > k:
            heapq.heappop(h)    # discard the smallest; keep the k largest
    return h[0]
```

Explanation and complexity: A min-heap of size k keeps the k largest values; its root is the k-th largest. O(n log k) time, O(k) space.

C34. Merge k sorted lists [Hard]

Merge k sorted lists into one sorted list. Example: [[1,4,5],[1,3,4],[2,6]] -> [1,1,2,3,4,4,5,6].

```
import heapq

def merge_k_sorted(lists):
    h = [(l[0], i, 0) for i, l in enumerate(lists) if l]
    heapq.heapify(h)
    out = []
    while h:
        v, i, j = heapq.heappop(h)
        out.append(v)
        if j + 1 < len(lists[i]):
            heapq.heappush(h, (lists[i][j + 1], i, j + 1))
    return out
```

Explanation and complexity: The heap always holds one candidate per list. O(N log k) for N total items. (heapq.merge(*lists) does exactly this.)

C35. K closest points to the origin [Medium]

Return the k points closest to (0, 0) (Euclidean), closest first. Example: ([[3,3],[5,-1],[-2,4]], 2) -> [[3,3],[-2,4]].

```
import heapq

def k_closest(points, k):
    return heapq.nsmallest(k, points, key=lambda p: p[0]**2 + p[1]**2)
```

Explanation and complexity: Compare squared distances (no sqrt needed). O(n log k).

C36. Search insert position [Easy]

In a sorted list, return the index where x is found or would be inserted. Example: ([1,3,5,6], 2) -> 1.

```
import bisect

def search_insert(a, x):
    return bisect.bisect_left(a, x)
```

Explanation and complexity: bisect_left is exactly the leftmost insertion point. O(log n).

C37. Count occurrences in a sorted list [Medium]

Count how many times x appears in a sorted list in O(log n). Example: ([1,2,2,2,3], 2) -> 3.

```
import bisect

def count_in_sorted(a, x):
    return bisect.bisect_right(a, x) - bisect.bisect_left(a, x)
```

Explanation and complexity: Right insertion point minus left insertion point = number of equal items.

C38. Longest increasing subsequence (length) [Hard]

Return the length of the longest strictly increasing subsequence. Example: [10,9,2,5,3,7,101,18] -> 4.

```
import bisect

def lis_length(a):
    tails = []
    for x in a:
        i = bisect.bisect_left(tails, x)
        if i == len(tails):
            tails.append(x)
        else:
            tails[i] = x
    return len(tails)
```

Explanation and complexity: tails stays sorted, so bisect_left finds where x fits. tails itself is not necessarily a real subsequence, but its length is correct. $O(n \log n)$.

C39. Minimum meeting rooms [Medium]

Given [start, end) intervals, return the minimum number of rooms needed. Example:

[[0,30],[5,10],[15,20]] -> 2.

```
import heapq

def min_meeting_rooms(intervals):
    ends = []
    for s, e in sorted(intervals):
        if ends and ends[0] <= s:
            heapq.heapreplace(ends, e)
        else:
            heapq.heappush(ends, e)
    return len(ends)
```

Explanation and complexity: Sort by start; the heap root is the room that becomes free earliest. $O(n \log n)$.

C40. Running median [Hard]

Implement MedianFinder with add(x) and median() for a stream of numbers. Example: add 1, add 2 -> median 1.5; add 3 -> median 2.0.

```
import heapq

class MedianFinder:
    def __init__(self):
        self.lo = []
        self.hi = []

    def add(self, x):
        heapq.heappush(self.lo, -x)
        heapq.heappush(self.hi, -heapq.heappop(self.lo))
        if len(self.hi) > len(self.lo):
            heapq.heappush(self.lo, -heapq.heappop(self.hi))

    def median(self):
        if len(self.lo) > len(self.hi):
            return float(-self.lo[0])
        return (-self.lo[0] + self.hi[0]) / 2
```

Explanation and complexity: Two heaps split the data around the median: add is $O(\log n)$, median is $O(1)$. A classic "heap of heaps" interview problem.

9. Interview Tricks

What interviewers *actually* listen for, and the small habits that turn a correct answer into a strong one.

9.1 Before you type

- **Clarify:** sorted? duplicates? negatives? empty input? case-sensitive? Must order be kept? In-place allowed?
- **Say the brute force first** (" $O(n^2)$ double loop"), then the improvement ("hash map gives $O(n)$ "). It shows you know what you are optimising.
- **Name the pattern:** "this is a frequency-count / two-sum / monotonic stack / BFS problem".
- **State complexity** for time *and* space before and after coding. Include the hidden cost of slicing, in list, string concatenation in a loop, and sorted.

9.2 Tool-choice reflexes

If you catch yourself writing...	Switch to...
<code>if x not in d: d[x] = 0</code>	<code>Counter / defaultdict(int) / d.get(x, 0)</code>
<code>if k not in d: d[k] = []</code>	<code>defaultdict(list) / d.setdefault(k, [])</code>
<code>x in some_list</code> inside a loop	convert to set first
<code>list.pop(0) / insert(0, x)</code>	deque
<code>for i in range(len(a))</code>	enumerate / zip / direct iteration
<code>sorted(...)[k]</code> for tiny k	<code>heapq.nsmallest(k, ...)</code>
hand-written binary search for a position	<code>bisect_left / bisect_right</code>
a loop that sets a flag then breaks	<code>any(...)</code> / <code>all(...)</code>
<code>s += ch</code> inside a big loop	collect in a list, then <code>''.join(parts)</code>
manual swap with a temp	<code>a, b = b, a</code>

9.3 The classic traps (know all of them)

Trap	The truth
<code>a = a.sort(), a = a.reverse(), a = a.append(x)</code>	in-place methods return None
<code>[[0]*3]*3</code>	three references to one inner list; use a comprehension
<code>list(set(a))</code>	order lost; <code>dict.fromkeys</code> keeps it
<code>round(2.5)</code>	2, banker's rounding
<code>any([]), all([])</code>	False, True
<code>max([])</code>	ValueError unless <code>default=</code>
<code>map/filter/zip</code> reused	one-shot iterators; second pass is empty
<code>isinstance(True, int)</code>	True; bool subclasses int
<code>is</code> vs <code>==</code>	identity vs equality; only use <code>is</code> with None
<code>d[k]</code> on <code>defaultdict</code>	creates the key just by reading it
<code>heapq</code> is a min-heap	negate for max; add tie-breaker to tuples
<code>sorted('abc')</code>	returns a list of characters
default arguments <code>def f(x, acc=[])</code>	the list is shared across calls; use None and create inside
lambda in a loop	late binding; <code>lambda i=i: ...</code>
<code>-7 // 2, -7 % 2</code>	-4 and 1 (floor); C++ gives -3 and -1
<code>bisect</code> on unsorted data	silently wrong

Trap	The truth
<code>s.find(x)</code> truthiness	<code>-1</code> is truthy
<code>(1)</code> vs <code>(1,)</code>	int vs one-element tuple
<code>sum(list_of_lists, [])</code>	works but $O(n^2)$
zip with unequal lengths	silently stops at the shortest

9.4 Phrases that impress

- "I'll use a hash map to trade $O(n)$ space for $O(n)$ time."
- "`deque.popleft()` is $O(1)$; `list.pop(0)` is $O(n)$ because every element shifts."
- "Python's sort is Timsort: $O(n \log n)$ worst case, stable, and $O(n)$ on nearly-sorted input."
- "A heap gives me the minimum in $O(1)$ and updates in $O(\log n)$; building it with `heapify` is $O(n)$."
- "`nlargest(k)` is $O(n \log k)$, better than sorting when k is much smaller than n ."
- "I'm using a generator expression so no intermediate list is built - $O(1)$ extra space."
- "`is` checks identity, `==` checks equality; I use `is` only for `None`."
- "Because the sort is stable, I can sort by the secondary key first and the primary key second."

9.5 Practice routine

- Do every **output question** cold: cover the answer, predict, then verify by running it.
- For each coding problem: write the brute force, then the pattern solution, then state both complexities.
- Re-solve problems 24-40 the next day **without** looking; retrieval practice beats rereading.
- Keep a personal "traps" list: every time Python surprises you, add one line to it.
- Move on to Part 5 once you can explain **why** each answer in sections 8.2 and 9.3 is what it is.